Trajectory-Level Risk Allocation for Multi-Horizon Online Conformal Prediction

Dependence Learning, Right-Censored Feedback, and Online Guarantees

One-page problem summary

Objective. At forecast origin k, issue H nested prediction sets $\{\mathcal{C}_{k,h}(\alpha_{k,h})\}_{h=1}^{H}$. Let

$$V_{k,h} := \mathbf{1}\{Y_{k+h} \notin \mathcal{C}_{k,h}(\alpha_{k,h})\}, \qquad Z_k := \max_{1\le h\le H} V_{k,h}.$$

The primary validity objective is whole-trajectory calibration,

$$\frac{1}{K}\sum_{k=1}^{K} Z_k \longrightarrow \beta,$$

while the primary efficiency objective is low regret relative to an oracle trajectory-risk allocator.

Outer calibration state and common risk resource. Each trajectory contributes exactly one binary outcome Z_k. Its label is processed when it becomes known: at the first violation if the trajectory fails, or at horizon H if it succeeds. A scalar state q_t carries the exact resolved-label accounting,

$$q_t = q_{t-1} + \eta \sum_{k\in\mathcal{R}_t} (\beta - Z_k).$$

On ordinary rounds, a predictable outer controller proposes a scalar trajectory-risk budget $\widetilde{b}_t \in [0,1]$ from the available feedback signal. The exact state q_t acts as a safety wrapper: outside fixed state thresholds the algorithm uses formal conservative/aggressive extreme actions, while inside the safe region it exposes $b_t = \widetilde{b}_t$ to the inner allocator. The clipped affine map $\widetilde{b}_t = \phi(q_t)$ is the simplest baseline. Thus β remains fixed, q_t carries validity accounting, and b_t is the resource seen by the optimizer.

Dependence-aware allocation. Given b_t, horizon-specific marginal miscoverage levels are selected by

$$\boxed{\boldsymbol{\alpha}_t \in \arg\max_{\boldsymbol{\alpha}\in\mathcal{A}} \sum_{h=1}^{H} U_{t,h}(\alpha_h) \quad \text{s.t.} \quad \widehat{R}_t(\boldsymbol{\alpha}) \le b_t.}$$

For a predictable fixed forest F_t,

$$\widehat{R}_t(\boldsymbol{\alpha}) = \sum_{h=1}^{H} \alpha_h - \sum_{e=(s,r)\in F_t} \widetilde{p}_{t,e}(\alpha_s, \alpha_r),$$

where $\widetilde{p}_{t,e}$ is a conservative lower certificate for simultaneous failure probability. The dependence term is therefore an allocation-dependent risk rebate, not an additional scalar budget.

Why right-censoring matters. A failed trajectory is known at its first violation, while a successful trajectory is known only after all H horizons survive. Nevertheless at most H issued trajectories are unresolved at any physical time. This yields the pathwise accounting bound

$$\left|\frac{1}{T}\sum_{k=1}^{T}(Z_k - \beta)\right| \le \frac{|q_T - q_0|}{\eta T} + \frac{H}{T}.$$

With extreme prediction-set safeguards, the latent state is uniformly bounded, so for fixed H the trajectory-calibration error is $\mathcal{O}(T^{-1})$.

Two performance criteria. All statistical, numerical, delay, forest-selection, and shape-approximation errors are treated as contributors to one of two headline quantities:

$$\boxed{\mathrm{Reg}_T^{\mathrm{opt}}} \qquad \text{and} \qquad \boxed{\mathrm{Cal}_T := \frac{1}{T}\sum_{k=1}^{T}(Z_k - \beta).}$$

The first measures efficiency relative to an oracle allocator under the same outer budget sequence; the second measures realized trajectory-level coverage. On the simultaneous dependence-certification event, stable dependence, bounded risk prices, and vanishing cumulative approximation error imply sublinear optimization regret. Shape loss is not a third metric: it is one possible contribution to $\mathrm{Reg}_T^{\mathrm{opt}}$.

Information flow

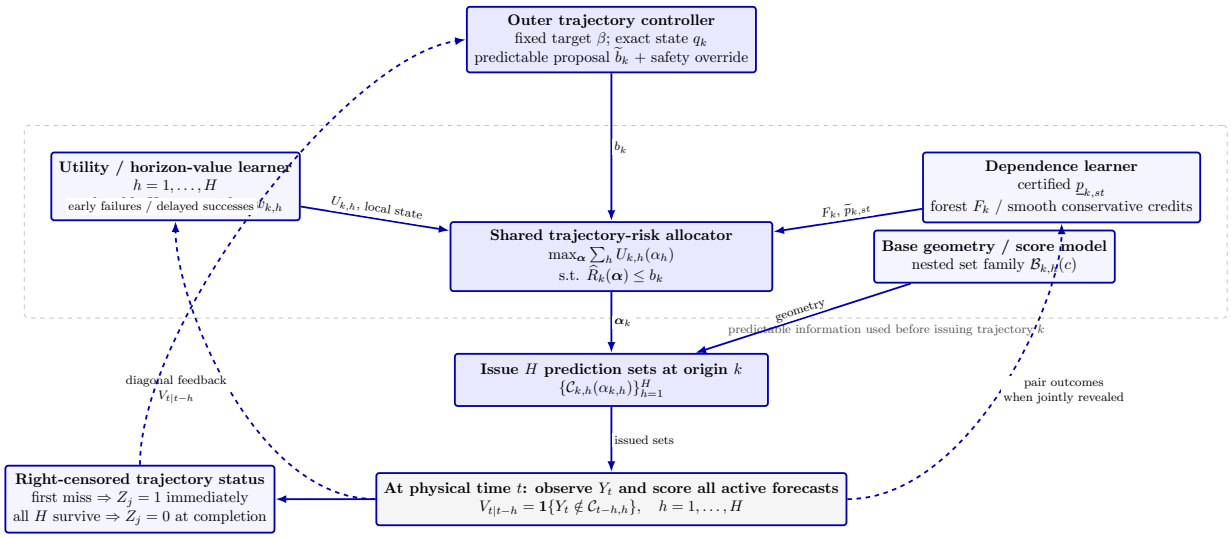


Figure 1: Information flow for the proposed multi-horizon online conformal architecture. A single physical observation resolves one horizon-specific error for each active forecast origin. These shared observations update horizon learners and dependence estimates, while trajectory-level failures are revealed early and successes only after the full horizon. The common risk allocator combines the safeguarded scalar budget, horizon utilities, and certified dependence credit before a warm-started SQP/RTI step issues the next trajectory.

1 Motivation

Consider a forecaster that, at physical time k, issues prediction sets for the next H outcomes. The hth set predicts Y_{k+h} and is denoted

$$\mathcal{C}_{k,h}(\alpha_{k,h}), \qquad h = 1, \dots, H,$$

where $\alpha_{k,h}$ is a horizon-specific marginal miscoverage allocation. Larger $\alpha_{k,h}$ typically permits a smaller prediction set, so marginal risk is a scarce resource that can be exchanged for efficiency.

The desired trajectory event is not an average of marginal errors. It is whole-trajectory containment. Define

$$V_{k,h} := \mathbf{1}\{Y_{k+h} \notin \mathcal{C}_{k,h}(\alpha_{k,h})\}, \qquad Z_k := \max_{1 \le h \le H} V_{k,h}. \tag{1}$$

Thus $Z_k = 1$ if at least one member of the H-step trajectory falls outside its issued prediction set. The long-run target is

$$\frac{1}{K} \sum_{k=1}^{K} Z_k \approx \beta, \tag{2}$$

where β is the desired trajectory-level miscoverage rate.

A uniform allocation $\alpha_{k,1} = \cdots = \alpha_{k,H}$ is generally wasteful. Different horizons have different predictive difficulty, different delays before their feedback is revealed, and different dependence relationships with the rest of the trajectory. The central design question is therefore:

> How should a common trajectory-level risk budget be allocated across horizons so that prediction sets are efficient, while the collection of allocations remains jointly compatible with the desired trajectory coverage?

2 Architecture and separation of roles

The framework separates four objects that should not be conflated.

Exact trajectory-level accounting. The desired trajectory miscoverage β is fixed. A scalar state q_t accumulates only resolved trajectory-level coverage credit/debt. This state is model free and is the object used in the pathwise calibration proof.

Ordinary-round actuation. A predictable feedback construction produces an ordinary-round control signal. A controller—for example a fixed-gain ACI update, a DtACI-style adaptive-gain scheme, or another stable scalar controller—maps that signal to a proposed budget $\widetilde{b}_t \in [0, 1]$. The exact state q_t supplies a separate safety override, producing the budget b_t that is actually exposed to the inner allocator. The baseline $\widetilde{b}_t = \phi(q_t)$ is recovered as a special case.

Horizon-specific value of risk. Each horizon h has a predictable utility function

$$U_{t,h} : \mathcal{A}_h \to \mathbb{R},$$

where $U_{t,h}(\alpha)$ quantifies the benefit of allocating marginal miscoverage α to that horizon. The utility may encode prediction-set width, volume, a downstream control objective, or a learned surrogate. In the proportional-fair specialization, its horizon weights may evolve slowly from set-size and first-failure information.

Trajectory-risk charge. The vector $\boldsymbol{\alpha}_t = (\alpha_{t,1}, \ldots, \alpha_{t,H})$ incurs a certified trajectory-risk charge

$$\widehat{R}_t(\boldsymbol{\alpha}_t).$$

This charge combines marginal risk with dependence information and determines which allocations are feasible:

$$\widehat{R}_t(\boldsymbol{\alpha}_t) \le b_t.$$

The separation is deliberate. The outer controller determines *how much* trajectory risk may be spent; the utility determines *where that risk is valuable*; the dependence certificate determines *how expensive* a candidate allocation is to whole-trajectory containment; and the exact state q_t remains the validity backbone.

3 General trajectory-risk allocation problem

Let

$$\mathcal{A} = \mathcal{A}_1 \times \cdots \times \mathcal{A}_H$$

be the feasible set of horizon allocations. The general inner problem is

$$\boxed{\begin{array}{ll} \max\limits_{\boldsymbol{\alpha}\in\mathcal{A}} & \sum\limits_{h=1}^{H} U_{k,h}(\alpha_h) \\ \text{s.t.} & \widehat{R}_k(\boldsymbol{\alpha}) \le b_k. \end{array}} \tag{3}$$

The formulation is intentionally agnostic about how $U_{k,h}$ and $\widehat{R}_k$ are learned. It only requires that they be determined from information available when the allocation is selected.

3.1 Dependence-aware trajectory risk

Suppose the marginal calibration layer supplies the horizon-wise charges α_h, and suppose that for selected horizon pairs (s,t) we have certified lower bounds

$$\underline{p}_{k,st}(\alpha_s, \alpha_t) \le \mathbb{P}(V_{k,s} = 1, V_{k,t} = 1) \tag{4}$$

under the conditioning/validity regime used by the calibration theorem. For a forest F_k on the horizon indices, Hunter's second-order union inequality gives the certified risk surrogate

$$\widehat{R}_{k,F_k}(\boldsymbol{\alpha}) := \sum_{h=1}^{H} \alpha_h - \sum_{(s,t)\in F_k} \underline{p}_{k,st}(\alpha_s, \alpha_t). \tag{5}$$

The first term is the Boole charge. The second term is certified overlap credit. Hence dependence learning does not create additional risk budget; it prevents the allocator from paying twice for failures that are known to occur together.

For the optimization layer, it is useful to treat the forest F_k as fixed while solving (3). It may be selected from predictable, certified information immediately before the allocation problem is solved. Jointly re-selecting the maximum-weight forest as a function of $\boldsymbol{\alpha}$ generally creates additional nonsmooth/nonconvex structure; Section 4 therefore freezes the forest inside each SQP solve.

3.2 A game-theoretic interpretation

Associate a nonnegative dual price λ_k with the common trajectory budget. The Lagrangian is

$$\mathcal{L}_k(\boldsymbol{\alpha}, \lambda) = -\sum_{h=1}^{H} U_{k,h}(\alpha_h) + \lambda\big(\widehat{R}_k(\boldsymbol{\alpha}) - b_k\big). \tag{6}$$

Each horizon behaves like an agent requesting risk from a common resource. The utility term rewards receiving additional marginal risk when doing so improves efficiency. The dual variable λ_k is the shadow price of trajectory risk. Dependence modifies the effective price paid by each horizon through its marginal contribution to $\widehat{R}_k$.

At an interior differentiable solution,

$$U'_{k,h}(\alpha_{k,h}) = \lambda_k \frac{\partial \widehat{R}_k}{\partial \alpha_h}(\boldsymbol{\alpha}_k). \tag{7}$$

Equation (7) captures the desired allocation principle: a horizon should receive more risk when the marginal utility is high or when an additional unit of its marginal risk contributes relatively little to whole-trajectory failure.

4 Fixed-forest inner optimization: SQP and an optional convex relaxation

The fixed-forest allocation problem is generally nonconvex. Coordinatewise monotonicity of an overlap surface does not imply concavity, so requiring a global concave minorant solely to obtain convexity can discard substantial dependence credit. The primary computational method in this paper is therefore a smooth conservative nonlinear program solved by warm-started sequential quadratic programming (SQP), or by one SQP real-time iteration when the sampling rate requires it.

Assumption 1 (Smooth certified SQP model)**.** *For round k:*

1. *$\mathcal{A}$ is a compact box with $0 < \alpha_{\min} \le \alpha_h \le \alpha_{\max} < 1$;*
2. *each $U_{k,h}$ is twice continuously differentiable on $\mathcal{A}_h$;*
3. *the forest F_k is selected predictably and held fixed during the nonlinear solve;*
4. *for every $e \in F_k$, the optimization surface $\widetilde{p}_{k,e}$ is twice continuously differentiable on the region queried by the solver and is conservative:*

$$\widetilde{p}_{k,e}(\boldsymbol{\alpha}_e) \le \underline{p}_{k,e}(\boldsymbol{\alpha}_e) \le p_{k,e}(\boldsymbol{\alpha}_e).$$

Define

$$\widetilde{R}_{k,F_k}(\boldsymbol{\alpha}) := \sum_{h=1}^{H} \alpha_h - \sum_{e \in F_k} \widetilde{p}_{k,e}(\boldsymbol{\alpha}_e). \tag{8}$$

The ordinary inner problem is

$$\boxed{\begin{aligned} \max_{\boldsymbol{\alpha} \in \mathcal{A}} \quad & \sum_{h=1}^{H} U_{k,h}(\alpha_h) \\ \text{s.t.} \quad & \widetilde{R}_{k,F_k}(\boldsymbol{\alpha}) \le b_k. \end{aligned}} \tag{9}$$

No concavity assumption is imposed on $\widetilde{p}_{k,e}$ in the primary method. A feasible SQP output is sufficient for validity because replacing the true overlap by a pointwise lower surface only increases the Hunter risk surrogate. Global optimality affects efficiency, not coverage.

In implementation, the previous primal-dual solution is used as the warm start, the forest and learned surfaces are frozen during the solve, and the returned allocation is checked against

$$\widetilde{R}_{k,F_k}(\boldsymbol{\alpha}_k) \le b_k$$

before the prediction sets are issued. If the final iterate violates the constraint beyond numerical tolerance, the round is repaired or rejected in favor of a certified feasible allocation.

Why freeze the forest? If

$$W_k(\boldsymbol{\alpha}) = \max_{F \in \mathcal{F}} \sum_{e \in F} \widetilde{p}_{k,e}(\boldsymbol{\alpha}_e),$$

then changing the maximizing forest as $\boldsymbol{\alpha}$ moves introduces a piecewise-defined objective/constraint surface and can change the active combinatorial structure inside the nonlinear solve. We therefore select F_k immediately before optimization and hold it fixed for that round. A suboptimal certified forest affects efficiency but not validity.

Optional convex relaxation. A globally convex inner problem remains available when desired. Suppose, in addition to the conditions above, that each $U_{k,h}$ is concave and each $\widetilde{p}_{k,e}$ is a concave conservative lower bound. Then $-\widetilde{p}_{k,e}$ is convex and $\widetilde{R}_{k,F_k}$ is convex. Hence

$$\{\boldsymbol{\alpha} \in \mathcal{A} : \widetilde{R}_{k,F_k}(\boldsymbol{\alpha}) \le b_k\}$$

is convex and maximizing the concave utility is a convex optimization problem. This relaxation is useful as a benchmark or fallback, but any additional loss of overlap credit caused by concavification is treated as approximation error in the efficiency analysis rather than as a validity requirement.

5 Specific instantiation: proportional-fair risk allocation

A particularly simple utility is weighted proportional fairness:

$$U_{k,h}(\alpha_h) = w_{k,h} \log \alpha_h, \qquad w_{k,h} > 0. \tag{10}$$

We restrict

$$0 < \alpha_{\min} \le \alpha_h \le \alpha_{\max} < 1$$

to avoid degeneracy and to encode any practical marginal-risk limits. The concrete inner problem is

$$\boxed{\begin{aligned} \max_{\boldsymbol{\alpha}} \quad & \sum_{h=1}^{H} w_{k,h} \log \alpha_h \\ \text{s.t.} \quad & \sum_{h=1}^{H} \alpha_h - \sum_{(s,t) \in F_k} \widetilde{p}_{k,st}(\alpha_s, \alpha_t) \le b_k, \\ & \alpha_{\min} \le \alpha_h \le \alpha_{\max}, \qquad h = 1, \dots, H. \end{aligned}} \tag{11}$$

In the primary implementation, (11) is a smooth fixed-forest nonlinear program solved by warm-started SQP. It becomes convex under the optional concave-overlap relaxation described in Section 4.

The logarithm provides an anti-starvation mechanism. Sending any horizon toward zero marginal risk incurs an increasingly large utility penalty. The weights $w_{k,h}$ encode the predictable horizon-specific value of additional marginal risk. They may be fixed exogenously or adapted online from prediction-set efficiency and first-failure information as described in Section 5.3.

5.1 Marginal risk price

Ignoring active box constraints, the KKT stationarity condition for (11) is

$$\frac{w_{k,h}}{\alpha_{k,h}} = \lambda_k \left[1 - \sum_{e \in F_k : e \ni h} \frac{\partial \widetilde{p}_{k,e}}{\partial \alpha_h} \right]. \tag{12}$$

The bracketed term is the marginal certified trajectory-risk charge for horizon h. Thus a horizon receives more marginal risk when either its weight $w_{k,h}$ is large or its additional failures overlap strongly with failures already represented by the forest.

Equation (12) gives a useful economic interpretation. The allocation does not ask whether horizon h is difficult in isolation. It asks how much efficiency is gained per unit of *incremental trajectory risk*. Two horizons can have identical marginal error behavior and nevertheless receive different allocations because their dependence with the rest of the trajectory differs.

5.2 Relation to direct set-size minimization

If a differentiable set-size function $S_{k,h}(\alpha)$ is available and is convex and decreasing in α, another principled choice is

$$U_{k,h}(\alpha) = -\omega_{k,h} S_{k,h}(\alpha).$$

Then (3) directly minimizes weighted total set size. The proportional-fair utility (10) is useful when a stable convex model of set size is unavailable or when explicit anti-starvation is desirable.

5.3 Adaptive horizon utility weights

The outer budget b_t controls how much total trajectory risk is available; the utility weights $\boldsymbol{w}_t = (w_{t,1}, \ldots, w_{t,H})$ control where that risk is valuable. The two feedback loops are intentionally separate. Calibration errors act on the outer budget, whereas efficiency and horizon fragility may act on the utility weights.

For proportional-fair utility,

$$U_t(\boldsymbol{\alpha}) = \sum_{h=1}^{H} w_{t,h} \log \alpha_h, \qquad \frac{\partial U_t}{\partial \alpha_h} = \frac{w_{t,h}}{\alpha_h}.$$

Thus increasing $w_{t,h}$ tends to allocate more marginal risk to horizon h. A horizon that is frequently the first location of trajectory failure should therefore not automatically receive more weight.

Static weights. Let $c_h > 0$ denote application-level horizon importance. The simplest choice is

$$w_{t,h} = \frac{c_h}{\sum_{j=1}^{H} c_j}. \tag{13}$$

This remains the default when no online utility adaptation is desired.

Marginal efficiency. Let $S_{t,h}(a)$ denote a predictable set-size or efficiency cost at marginal miscoverage a; examples include interval width, set volume, or an application-specific burden. Define

$$m_{t,h}(a) := -\frac{\partial S_{t,h}(a)}{\partial a} \geq 0. \tag{14}$$

If an analytic derivative is unavailable, estimate it on the marginal-risk grid by a centered finite difference or a smooth interpolant. Matching the local slope of $-c_h S_{t,h}(\alpha_h)$ to the log-utility slope gives the raw efficiency target

$$\boxed{w_{t,h}^{\mathrm{eff}} := c_h \alpha_{t-1,h} \widehat{m}_{t,h}, \qquad \widehat{m}_{t,h} := m_{t,h}(\alpha_{t-1,h}).} \tag{15}$$

Thus the numerically convenient log utility can locally track the direct set-size objective while retaining its anti-starvation behavior.

Optional failure-aware weighting. Let τ_k denote the first-failure horizon, with $\tau_k = \infty$ for a successful trajectory; Section 7 formalizes this quantity. A censoring-safe recent estimate of first-failure mass uses only fully resolved origins:

$$\widehat{f}_{t,h}^{(W)} := \frac{1}{W} \sum_{k=t-H-W+1}^{t-H} \mathbf{1}\{\tau_k = h\}. \tag{16}$$

Because every origin in this window is at least H steps old, early failures and delayed successes are treated symmetrically.

When failure-location information is used in the utility layer, a simple raw target is

$$\widetilde{w}_{t,h}^{\mathrm{tar}} := \frac{w_{t,h}^{\mathrm{eff}} + \varepsilon_m}{\varepsilon_f + \widehat{f}_{t,h}}, \qquad \varepsilon_m, \varepsilon_f > 0, \tag{17}$$

followed by normalization across horizons. This is an efficiency heuristic, not a coverage requirement: $\widehat{f}_{t,h}$ describes where additional risk has recently been fragile, whereas $\widehat{m}_{t,h}$ describes where additional risk is useful. Either factor may be omitted in an ablation.

RTI-compatible temporal smoothing. Let $\boldsymbol{w}_t^{\mathrm{tar}}$ denote the normalized target produced by the chosen weighting rule. To avoid moving the SQP optimum abruptly, first apply an exponential update

$$\boldsymbol{w}_{t+1}^{\mathrm{ema}} = (1 - \zeta_w)\boldsymbol{w}_t + \zeta_w \boldsymbol{w}_t^{\mathrm{tar}}, \qquad 0 < \zeta_w \leq 1, \tag{18}$$

and then project onto

$$\mathcal{W} := \left\{ \boldsymbol{w} \in \mathbb{R}^H : \sum_{h=1}^{H} w_h = 1,\ w_{\min} \leq w_h \leq w_{\max} \right\}. \tag{19}$$

For an RTI implementation, additionally impose

$$\|\boldsymbol{w}_{t+1} - \boldsymbol{w}_t\|_2 \leq \delta_w. \tag{20}$$

Equivalently,

$$\boxed{\boldsymbol{w}_{t+1} = \Pi_{\mathcal{W} \cap B_2(\boldsymbol{w}_t, \delta_w)}\big(\boldsymbol{w}_{t+1}^{\mathrm{ema}}\big).} \tag{21}$$

For the minimization form

$$f_t(\boldsymbol{\alpha}) := -\sum_{h=1}^{H} w_{t,h} \log \alpha_h,$$

we have

$$\nabla f_t(\boldsymbol{\alpha}) = -\big(w_{t,h}/\alpha_h\big)_{h=1}^{H}, \qquad \nabla^2 f_t(\boldsymbol{\alpha}) = \operatorname{diag}\left(\frac{w_{t,1}}{\alpha_1^2}, \ldots, \frac{w_{t,H}}{\alpha_H^2}\right).$$

Hence, for $\alpha_h \geq \alpha_{\min}$,

$$\|\nabla f_{t+1}(\boldsymbol{\alpha}) - \nabla f_t(\boldsymbol{\alpha})\|_2 \leq \frac{\|\boldsymbol{w}_{t+1} - \boldsymbol{w}_t\|_2}{\alpha_{\min}}, \tag{22}$$

and

$$\|\nabla^2 f_{t+1}(\boldsymbol{\alpha}) - \nabla^2 f_t(\boldsymbol{\alpha})\|_2 \leq \frac{\|\boldsymbol{w}_{t+1} - \boldsymbol{w}_t\|_\infty}{\alpha_{\min}^2}. \tag{23}$$

Thus ζ_w and δ_w directly control the objective perturbation seen by a warm-started SQP/RTI step. A useful summary quantity is

$$V_T^w := \sum_{t=1}^{T-1} \|\boldsymbol{w}_{t+1} - \boldsymbol{w}_t\|_2.$$

The round-t inner problem is therefore

$$\boxed{\boldsymbol{\alpha}_t \in \operatorname*{arg\,max}_{\boldsymbol{\alpha} \in \mathcal{A}_t} \left\{ \sum_{h=1}^{H} w_{t,h} \log \alpha_h : \widetilde{R}_t(\boldsymbol{\alpha}) \leq b_t \right\}.} \tag{24}$$

If the oracle comparator uses the same predictable realized sequence $\{\boldsymbol{w}_t\}$, weight adaptation is part of the round utility and creates no additional regret term. Comparing instead with a different latent utility sequence would require an additional utility-learning term.

6 Online Learning of Pairwise Dependence

The dependence module required by the trajectory-risk allocator is a direct application of the delayed bivariate martingale-CDF theory developed in the companion technical note. We therefore state only the specialization needed here and do not repeat its proofs.

For horizon h, let

$$V_{k,h}(a) := \mathbf{1}\left\{Y_{k+h} \notin \mathcal{C}_{k,h}(a)\right\}.$$

For an edge $e = (s,t)$, define

$$X_{k,e}(a,b) := V_{k,s}(a) V_{k,t}(b), \qquad p_{k,e}(a,b) := \mathbb{E}[X_{k,e}(a,b) \mid \mathcal{F}_k].$$

Thus $p_{k,e}(a,b)$ is the conditional probability that horizons s and t fail simultaneously at marginal risk levels (a,b). This is the dependence quantity that enters the Hunter forest bound.

Assume the prediction-set families are nested in the marginal miscoverage level. After Y_{k+h} is observed, define the critical level

$$A_{k,h}^\star := \sup\left\{a : Y_{k+h} \in \mathcal{C}_{k,h}(a)\right\}.$$

Up to the chosen boundary convention,

$$V_{k,h}(a) = \mathbf{1}\{A_{k,h}^\star < a\}.$$

Hence a resolved edge observation

$$B_{k,e} := \left(A^\star_{k,s}, A^\star_{k,t}\right)$$

determines

$$X_{k,e}(a,b) = \mathbf{1}\left\{A^\star_{k,s} < a,\ A^\star_{k,t} < b\right\}$$

simultaneously for every queried pair (a,b). Thus one resolved critical pair updates the entire empirical bivariate CDF surface; no exploration over grid points is required.

Let

$$\mathcal{G} = \{a_1, \ldots, a_M\}, \qquad \mathcal{Q} := \mathcal{G}^2, \qquad Q := |\mathcal{Q}| = M^2,$$

and let $\mathcal{E}$ be the candidate edge set from which the predictable forest F_k may be selected.

If horizon h is fully revealed after delay d_h, then edge $e = (s,t)$ is fully revealed after

$$d_e := \max\{d_s, d_t\}. \tag{25}$$

In the standard horizon-indexed setting $d_h = h$, so for $s < t$,

$$d_e = t.$$

At physical time k, let $\mathcal{I}_{k,e}$ denote the set of origins whose edge-e observations have fully resolved, and let

$$N_{k,e} := |\mathcal{I}_{k,e}|.$$

For $g = (a,b) \in \mathcal{Q}$, define

$$\widehat{p}_{k,e}(g) := \frac{1}{N_{k,e}} \sum_{j \in \mathcal{I}_{k,e}} X_{j,e}(g),$$

and define the corresponding average conditional target

$$\bar{p}_{k,e}(g) := \frac{1}{N_{k,e}} \sum_{j \in \mathcal{I}_{k,e}} p_{j,e}(g).$$

Theorem 1 (Delayed bivariate dependence certificate)**.** *Assume that the same finite query grid $\mathcal{Q}$ is used for every candidate edge, and define*

$$C_{\mathcal{E}} := Q \sum_{e \in \mathcal{E}} d_e. \tag{26}$$

Fix $\delta_{\mathrm{dep}} \in (0,1)$. Then the delayed bivariate martingale-CDF result implies that, with probability at least $1 - \delta_{\mathrm{dep}}$, simultaneously for every $e \in \mathcal{E}$, every physical time k with $N_{k,e} > 0$, and every $g \in \mathcal{Q}$,

$$\boxed{|\widehat{p}_{k,e}(g) - \bar{p}_{k,e}(g)| \le E_{k,e},} \tag{27}$$

where

$$\boxed{E_{k,e} := \sqrt{\frac{d_e}{2N_{k,e}} \log\left(\frac{\pi^2 C_{\mathcal{E}} N_{k,e}^2}{3\delta_{\mathrm{dep}}}\right)}.} \tag{28}$$

No independence, mixing, or stationarity assumption across forecast origins or across edges is required.

If all unordered horizon pairs are admissible and $d_h = h$, then exactly $h-1$ edges have delay h. Therefore

$$\sum_{e\in\mathcal{E}} d_e = \sum_{h=2}^{H} h(h-1) = \frac{H(H-1)(H+1)}{3}, \tag{29}$$

and hence

$$\boxed{C_{\mathcal{E}} = Q\frac{H(H-1)(H+1)}{3}.} \tag{30}$$

Because the confidence event is established over the full candidate edge set before the forest is selected, any predictable data-dependent forest

$$F_k \subseteq \mathcal{E}$$

is automatically covered by the same event.

Theorem 1 controls the empirical CDF relative to the running-average conditional surface $\bar{p}_{k,e}$. This distinction matters: without additional temporal regularity, historical data cannot provide a nontrivial distribution-free certificate for an arbitrarily changing current surface $p_{k,e}$.

6.1 Stable dependence

Suppose that for every candidate edge there exists a fixed conditional overlap surface p_e such that

$$p_{k,e}(g) = p_e(g) \qquad \text{for all } k,\ e\in\mathcal{E},\ g\in\mathcal{Q}. \tag{31}$$

Then

$$\bar{p}_{k,e}(g) = p_e(g),$$

and Theorem 1 gives the predictable lower certificate

$$\underline{p}_{k,e}(g) := \left[\widehat{p}_{k,e}(g) - E_{k,e}\right]_+. \tag{32}$$

On the same event,

$$\boxed{\underline{p}_{k,e}(g) \le p_e(g)} \tag{33}$$

simultaneously for every candidate edge, grid point, and physical time.

Moreover, since

$$|\widehat{p}_{k,e}(g) - p_e(g)| \le E_{k,e},$$

the lower certificate satisfies

$$\boxed{0 \le p_e(g) - \underline{p}_{k,e}(g) \le 2E_{k,e}.} \tag{34}$$

Since $p_e(a,b)$ is coordinatewise nondecreasing, the grid certificate may be extended conservatively by

$$\underline{p}^{\mathrm{mono}}_{k,e}(a,b) := \max_{\substack{a_i\le a\\ a_j\le b}} \underline{p}_{k,e}(a_i,a_j). \tag{35}$$

The primary SQP implementation uses a predictable smooth conservative surface $\widetilde{p}_{k,e}$ satisfying

$$\widetilde{p}_{k,e}(a,b) \le \underline{p}^{\mathrm{mono}}_{k,e}(a,b) \tag{36}$$

on the optimization region. Concavity is not required. Conservative interpolation or smoothing may be used provided the one-sided inequality is verified. If the optional convex relaxation of Section 4 is used instead, $\widetilde{p}_{k,e}$ may additionally be replaced by a concave minorant. Any extra interpolation, smoothing, or shape loss is tracked as an efficiency-approximation term rather than a validity condition.

6.2 Changing dependence

The stable-dependence assumption is not required for adaptive estimation. Suppose instead that $p_{k,e}$ may vary with k. For a finite dictionary of window lengths

$$1 \le W_1 < \cdots < W_J,$$

define the current-CDF squared estimation risk

$$\mathcal{R}_{k,e}(q) := \frac{1}{Q} \sum_{g \in \mathcal{Q}} \left(q(g) - p_{k,e}(g)\right)^2. \tag{37}$$

Define the one-step lower-orthant variation of edge e by

$$v_{u,e} := \sup_{g \in \mathcal{Q}} \left|p_{u+1,e}(g) - p_{u,e}(g)\right|, \tag{38}$$

and for $a < b$ define

$$V_{a:b,e} := \sum_{u=a}^{b-1} v_{u,e}. \tag{39}$$

Let $I = [r, s]$ be a physical-time interval after burn-in and let

$$n_I := |I|.$$

For window j, define the enlarged-interval variation

$$\mathcal{V}_{I,e,j} := V_{r-d_e-W_j+1:s,e}, \tag{40}$$

and its average age

$$A_{j,e} := d_e + \frac{W_j - 1}{2}. \tag{41}$$

Let

$$n_{\max,e} := \left\lceil \frac{T_{\max}}{d_e} \right\rceil, \tag{42}$$

and let

$$d_{I,e} \le \min\{d_e, n_I\}$$

denote the number of edge-e residue classes represented in I.

Theorem 2 (Variation-adaptive edgewise CDF estimation)**.** *Run the residue-wise fixed-share aggregation scheme of the companion bivariate-CDF result for edge e, with*

$$0 < \eta \le \frac{1}{2}.$$

Then, for every physical-time interval $I = [r, s]$ after burn-in,

$$\boxed{\sum_{k \in I} \mathbb{E}\left[\mathcal{R}_{k,e}\left(\widehat{p}^{\,\mathrm{mix}}_{k,e}\right)\right] \le \frac{1}{1-\eta}\left[\min_{j \in [J]}\left\{\frac{n_I d_e}{2W_j} + 2A_{j,e}\mathbb{E}[\mathcal{V}_{I,e,j}]\right\} + \frac{d_{I,e}[\log(n_{\max,e}J) + 2]}{\eta} + \frac{\eta n_I}{4}\right].} \tag{43}$$

If there exists a deterministic pathwise budget $V_{I,e}$ satisfying

$$\mathcal{V}_{I,e,j} \le V_{I,e} \qquad \text{for every } j \in [J], \tag{44}$$

then

$$\boxed{\sum_{k\in I}\mathbb{E}\big[\mathcal{R}_{k,e}\left(\widehat{p}^{\mathrm{mix}}_{k,e}\right)\big] \le \frac{1}{1-\eta}\left[\min_{j\in[J]}\left\{\frac{n_I d_e}{2W_j} + 2A_{j,e}V_{I,e}\right\} + \frac{d_{I,e}[\log(n_{\max,e}J)+2]}{\eta} + \frac{\eta n_I}{4}\right].} \tag{45}$$

The fixed-share update does not require prior knowledge of $V_{I,e}$.

The left-hand side of (43) has a direct truth-oracle interpretation. Since

$$\mathcal{R}_{k,e}(p_{k,e}) = 0,$$

it is the cumulative expected squared CDF-estimation error relative to a clairvoyant oracle that knows the current conditional bivariate CDF $p_{k,e}$ at every origin.

The first term on the right-hand side of (45),

$$\frac{n_I d_e}{2W_j} + 2A_{j,e}V_{I,e},$$

is the estimation cost of using the fixed memory scale W_j. The remaining terms quantify the additional cost of adaptively learning the appropriate memory scale.

Corollary 1 (Explicit variation-budget rate)**.** *Suppose*

$$0 < V_{I,e} < \infty$$

and define

$$W^{\star}_{I,e} := \sqrt{\frac{n_I d_e}{2V_{I,e}}}. \tag{46}$$

If the window dictionary contains some W_j satisfying

$$\frac{1}{2}W^{\star}_{I,e} \le W_j \le 2W^{\star}_{I,e}, \tag{47}$$

then

$$\boxed{\min_{j\in[J]}\left\{\frac{n_I d_e}{2W_j} + 2A_{j,e}V_{I,e}\right\} \le \frac{5}{2}\sqrt{\frac{n_I d_e V_{I,e}}{2}} + (2d_e - 1)V_{I,e}.} \tag{48}$$

Consequently,

$$\boxed{\sum_{k\in I}\mathbb{E}\big[\mathcal{R}_{k,e}\left(\widehat{p}^{\mathrm{mix}}_{k,e}\right)\big] \le \frac{1}{1-\eta}\left[\frac{5}{2}\sqrt{\frac{n_I d_e V_{I,e}}{2}} + (2d_e - 1)V_{I,e} + \frac{d_{I,e}[\log(n_{\max,e}J)+2]}{\eta} + \frac{\eta n_I}{4}\right].} \tag{49}$$

Theorem 2 is an estimation guarantee for the current conditional overlap surface. It does not itself provide a one-sided current-time confidence certificate under arbitrary nonstationarity. Such a certificate requires additional temporal regularity, such as a valid one-step drift envelope. This distinction is the same as in the companion bivariate-CDF analysis: the adaptive-regret result controls estimation quality, whereas the δ_{dep} concentration result controls statistical uncertainty.

6.3 Interface with the trajectory-risk allocator

The inner allocation problem receives a predictable overlap-credit surface $\widetilde{p}_{k,e}$ and uses

$$\widehat{R}_k(\boldsymbol{\alpha}) = \sum_{h=1}^{H} \alpha_h - \sum_{e \in F_k} \widetilde{p}_{k,e}(\boldsymbol{\alpha}_e). \tag{50}$$

Under stable dependence, on the simultaneous event of Theorem 1, the certified construction satisfies

$$\widetilde{p}_{k,e}(a,b) \le p_e(a,b)$$

for every selected edge and queried risk pair. Hence the dependence module can only subtract overlap credit that has been statistically certified.

If the comparator is allowed to choose the best true forest rather than the same predictable forest F_k, define the forest-selection gap

$$S_k^{\text{forest}}(\boldsymbol{\alpha}) := \max_{F} \sum_{e \in F} p_{k,e}(\boldsymbol{\alpha}_e) - \sum_{e \in F_k} p_{k,e}(\boldsymbol{\alpha}_e). \tag{51}$$

This is an optimization-comparator term rather than a separate statistical performance criterion.

The bivariate dependence result therefore supplies the statistical module needed by the trajectory-risk allocator. A single resolved critical-level pair updates the entire pairwise overlap surface; the reveal delay enters edge-specifically through

$$d_e = \max\{d_s, d_t\};$$

one global confidence event simultaneously covers every candidate edge from which the online forest may be selected; and the variation-adaptive estimator provides an explicit inequality for cumulative current-CDF estimation error when the dependence structure changes over time.

The remaining feedback issue is different. Trajectory-level success is not known until the trajectory has completed, whereas a failure may sometimes be known earlier. We formalize that right-censored trajectory-feedback structure next.

7 Right-Censored Trajectory Feedback

Trajectory-level feedback arrives asymmetrically. A failure becomes known as soon as the first prediction set is violated, whereas successful whole-trajectory containment can only be confirmed after the full horizon has elapsed. This section formalizes that information structure and derives the exact accounting identity used by the outer controller.

Throughout this section, k indexes a trajectory origin and t denotes physical time. At physical time t, newly revealed labels are processed first; the updated state is then used to choose the budget and issue the trajectory originating at t.

7.1 First failure and reveal time

For trajectory origin k, define

$$V_{k,h} := \mathbf{1}\{Y_{k+h} \notin \mathcal{C}_{k,h}\}, \qquad h = 1, \dots, H, \tag{52}$$

and

$$Z_k := \max_{1 \le h \le H} V_{k,h}. \tag{53}$$

The fixed target is

$$\frac{1}{K}\sum_{k=1}^{K} Z_k \longrightarrow \beta.$$

Let

$$\tau_k := \inf\{h \in \{1,\dots,H\} : V_{k,h} = 1\},$$

with $\tau_k = \infty$ if the complete trajectory remains contained. Define the reveal time

$$\rho_k := k + \begin{cases} \tau_k, & \tau_k \le H, \\ H, & \tau_k = \infty. \end{cases} \tag{54}$$

Thus the realized reveal delay is

$$d_k := \rho_k - k = \min\{\tau_k, H\} \le H.$$

A failed trajectory is therefore recorded at its first violation; it is not counted again at later horizons.

At physical time t, let

$$\mathcal{R}_t := \{k : \rho_k = t\} \tag{55}$$

denote the trajectories whose binary labels become known. Later observations from an already failed trajectory may still update horizon-specific or dependence learners, but they do not generate another trajectory-level error.

7.2 Resolved and pending trajectories

At physical time T, define

$$\mathcal{D}_T := \{k \le T : \rho_k \le T\}, \qquad \mathcal{P}_T := \{k \le T : \rho_k > T\}.$$

Because one trajectory is issued per time step and every label resolves within H steps,

$$|\mathcal{P}_T| \le H. \tag{56}$$

If

$$F_T := \sum_{k \in \mathcal{D}_T} Z_k$$

is the number of already revealed failures, then the total number of failures among trajectories issued through T satisfies the deterministic bracket

$$\boxed{F_T \le \sum_{k=1}^{T} Z_k \le F_T + |\mathcal{P}_T|.} \tag{57}$$

Hence uncertainty caused solely by unresolved trajectory labels is at most H/T. No independence, exchangeability, or censoring model is needed for this statement.

7.3 Exact trajectory-level accounting

Maintain an unprojected scalar accounting state $q_t \in \mathbb{R}$. After processing the labels revealed at physical time t, update

$$q_t = q_{t-1} + \eta \sum_{k \in \mathcal{R}_t} (\beta - Z_k), \qquad \eta > 0. \tag{58}$$

Every trajectory contributes exactly once. Summing through physical time T gives

$$q_T - q_0 = \eta \sum_{k \in \mathcal{D}_T} (\beta - Z_k). \tag{59}$$

Therefore

$$\sum_{k=1}^{T} (Z_k - \beta) = \frac{q_0 - q_T}{\eta} + \sum_{k \in \mathcal{P}_T} (Z_k - \beta), \tag{60}$$

and, since $|Z_k - \beta| \le 1$,

$$\boxed{\left| \frac{1}{T} \sum_{k=1}^{T} (Z_k - \beta) \right| \le \frac{|q_T - q_0|}{\eta T} + \frac{H}{T}.} \tag{61}$$

Proposition 1 (Calibration from a sublinear accounting state)**.** *If $|q_T| = o(T)$, then*

$$\frac{1}{T} \sum_{k=1}^{T} Z_k \longrightarrow \beta.$$

In particular, uniform boundedness of q_t is sufficient.

Equation (61) is the key separation: right censoring contributes only a finite boundary term, while the controller is responsible for keeping the latent accounting state sublinear.

7.4 Predict-and-correct feedback signals

The exact state q_t intentionally waits for resolved labels. For actuation, however, that reveal stream can be temporally asymmetric: failures may arrive early, while successes are not certified until horizon H. We therefore separate *validity accounting* from the construction of an ordinary-round control signal.

Let $\widehat{z}_{k,t} \in [0, 1]$ be a predictable provisional value for the terminal trajectory label Z_k at physical time t while trajectory k is pending. When the trajectory resolves, set

$$\widehat{z}_{k,\rho_k} = Z_k.$$

The choice of $\widehat{z}_{k,t}$ determines only when feedback is delivered. Four useful constructions are:

1. **Reveal-only feedback.** Set $\widehat{z}_{k,t} = \beta$ while the trajectory is unresolved. The issue-time contribution is zero and the full update is deferred until Z_k resolves. This is exactly the original accounting convention.

2. **Pending-is-valid feedback.** Set $\widehat{z}_{k,t} = 0$ while no violation has occurred. A new trajectory receives immediate success credit; if it later fails, that credit is corrected in one step. This removes delayed positive credit but creates larger failure corrections.

3. **Recent completed-window probability.** For a window W, use only fully resolved origin cohorts,

$$\widehat{\pi}_t^{(W)} := \frac{1}{W} \sum_{k=t-H-W+1}^{t-H} Z_k, \tag{62}$$

after burn-in, and initialize a new trajectory with $\widehat{z}_{t,t} = \widehat{\pi}_t^{(W)}$. Because all origins in the window are at least H steps old, the estimate is not distorted by the fact that failures reveal earlier than successes. The window may be fixed or chosen adaptively by the same multi-window regret machinery used elsewhere in the paper.

4. **Learned soft probability.** Use a predictable model

$$\widehat{z}_{k,t} = \widehat{\mathbb{P}}(Z_k = 1 \mid \mathcal{F}_t, \boldsymbol{\alpha}_k, \text{observed trajectory prefix}) . \tag{63}$$

One optional parameterization models the first-failure hazard

$$\vartheta_{k,h} := \mathbb{P}(\tau_k = h \mid \tau_k \geq h, \mathcal{F}_k, \boldsymbol{\alpha}_k)$$

and combines estimated hazards through

$$\widehat{\mathbb{P}}(Z_k = 1 \mid \mathcal{F}_k, \boldsymbol{\alpha}_k) = 1 - \prod_{h=1}^{H} (1 - \widehat{\vartheta}_{k,h}).$$

The hazard model is optional; the predict-and-correct identity below does not depend on its correctness.

Unified predict-and-correct update. When trajectory k is issued, book the provisional contribution

$$\eta_s(\beta - \widehat{z}_{k,k}). \tag{64}$$

Whenever its provisional value changes while it remains pending, book

$$\eta_s(\widehat{z}_{k,t-1} - \widehat{z}_{k,t}). \tag{65}$$

At resolution the final provisional value is Z_k, so every trajectory satisfies the exact telescoping identity

$$\boxed{\eta_s(\beta - \widehat{z}_{k,k}) + \eta_s \sum_{t=k+1}^{\rho_k} (\widehat{z}_{k,t-1} - \widehat{z}_{k,t}) = \eta_s(\beta - Z_k).} \tag{66}$$

Thus a poor provisional model can hurt transient smoothness but cannot change the eventual trajectory-level contribution.

Let s_t denote the cumulative predict-and-correct actuation state after all updates through physical time t. Then

$$s_T - s_0 = \eta_s \sum_{k=1}^{T} (\beta - \widehat{z}_{k,T}), \tag{67}$$

where $\widehat{z}_{k,T} = Z_k$ for resolved trajectories. Therefore

$$\boxed{\sum_{k=1}^{T} (Z_k - \beta) = -\frac{s_T - s_0}{\eta_s} + \sum_{k \in \mathcal{P}_T} (Z_k - \widehat{z}_{k,T}).} \tag{68}$$

Consequently,

$$\boxed{\left|\frac{1}{T}\sum_{k=1}^{T}(Z_k-\beta)\right| \le \frac{|s_T-s_0|}{\eta_s T} + \frac{1}{T}\left|\sum_{k\in\mathcal{P}_T}(Z_k-\widehat{z}_{k,T})\right| \le \frac{|s_T-s_0|}{\eta_s T} + \frac{H}{T}.} \tag{69}$$

The sharper middle expression shows why soft prediction can stabilize the boundary term, but the paper's validity theorem continues to rely on the exact resolved-label state q_t, not on the accuracy of $\widehat{z}_{k,t}$.

7.5 Accounting versus actuation

The exact state q_t, the provisional feedback signal, and the risk budget have different roles. The state q_t is the model-free trajectory-level accounting variable used by the pathwise validity theorem. The predict-and-correct layer produces a smoother ordinary-round signal, and a scalar controller converts that signal into a proposed budget $\widetilde{b}_t \in [0,1]$.

Write the net predict-and-correct increment at physical time t as g_t, so that

$$s_t = s_{t-1} + \eta_s g_t.$$

A fixed-gain ACI-style controller [2] has the generic form

$$\widetilde{b}_{t+1} = \Pi_{[0,1]}\left(\widetilde{b}_t + \gamma g_t\right). \tag{70}$$

A DtACI-style controller runs several gains $\gamma_1,\dots,\gamma_J$ in parallel,

$$b_{t+1,j} = \Pi_{[0,1]}(b_{t,j} + \gamma_j g_t),$$

and combines them with predictable adaptive weights. Literal DtACI uses an expert loss derived from a scalar critical conformal level [3]. In the present trajectory-allocation problem, that reduction is exact only if varying the scalar budget induces a nested family of trajectory predictions. Without that nestedness property, the same software pattern should be described as *DtACI-style adaptive gain aggregation* rather than as the original DtACI theorem. A conformal PID controller can be inserted at the same interface without changing the inner allocation module [4].

The pathwise safeguard is deliberately controller agnostic. Choose finite state thresholds $q^- < q^+$ and set

$$\boxed{b_t = \begin{cases} 0, & q_t \le q^-, \\ 1, & q_t \ge q^+, \\ \widetilde{b}_t, & q^- < q_t < q^+. \end{cases}} \tag{71}$$

The boundary values are implemented by the formal extreme actions defined in Section 8. Hence the ordinary controller may be ACI, DtACI-style, PID, or another predictable scalar policy; the exact state q_t remains the safety and calibration backbone.

The information flow is therefore

$$\text{resolved } Z_k \longrightarrow q_t \longrightarrow \text{safety override},$$

together with

$$\text{pending/resolved feedback} \longrightarrow g_t \longrightarrow \widetilde{b}_t,$$

and finally

$$(b_t, \boldsymbol{w}_t, \widetilde{R}_t) \longrightarrow \text{SQP/RTI} \longrightarrow \boldsymbol{\alpha}_t.$$

Algorithm 1 Coverage-aware trajectory-risk allocation at physical time t

Observe Y_t and score all active forecasts that resolve at time t.
Update horizon-specific learners and all newly available dependence statistics/certificates.
Form $\mathcal{R}_t = \{k : \rho_k = t\}$ and update the exact state q_t by (58).
Refresh provisional labels for previously issued pending trajectories; pass the resulting net signal to the ordinary outer controller.
Update the predictable utility weights $\boldsymbol{w}_t$ from information available before issuing trajectory t.

if $q_t \le q^-$ **then**
  Set $b_t = 0$ and issue the conservative extreme action $\mathcal{C}_{t,h} = \mathcal{Y}$ for all h.
else if $q_t \ge q^+$ **then**
  Set $b_t = 1$ and issue the aggressive extreme action, e.g. $\mathcal{C}_{t,1} = \varnothing$.
else
  Set $b_t = \widetilde{b}_t$, freeze a predictable forest F_t, and perform the warm-started SQP/RTI solve of (73).
  Verify $\widehat{R}_t(\boldsymbol{\alpha}_t) \le b_t$ to numerical tolerance, repair if necessary, and issue the resulting H sets.
end if
After $\boldsymbol{\alpha}_t$ is fixed, initialize the provisional signal for the newly issued trajectory; this contribution affects subsequent controller updates, not the already chosen round-t budget.

8 Coverage-Aware Risk Allocation Game

The outer state determines how much trajectory risk is available; the inner game determines how that scalar resource should be distributed across horizons. We now close both theoretical loops: pathwise trajectory calibration and efficiency relative to an oracle allocator.

8.1 Safeguarded outer budget and dependence-aware allocation

Let $\widetilde{b}_t \in [0,1]$ denote the predictable ordinary-round budget proposal produced by the chosen outer controller. The exact accounting state q_t is used only as the safety wrapper in (71). A convenient baseline is the clipped affine proposal

$$\widetilde{b}_t = \phi(q_t) := \mathrm{proj}_{[0,1]}(\beta + \kappa q_t), \qquad \kappa > 0, \tag{72}$$

but the ordinary controller may instead be ACI, DtACI-style adaptive gain aggregation, conformal PID, or another predictable scalar policy.

On ordinary rounds, the allocator solves

$$\boxed{\begin{aligned} \max_{\boldsymbol{\alpha} \in \mathcal{A}_t} \quad & U_t(\boldsymbol{\alpha}) := \sum_{h=1}^{H} U_{t,h}(\alpha_h) \\ \text{s.t.} \quad & \widehat{R}_t(\boldsymbol{\alpha}) := \sum_{h=1}^{H} \alpha_h - \sum_{e \in F_t} \widetilde{p}_{t,e}(\boldsymbol{\alpha}_e) \le b_t. \end{aligned}} \tag{73}$$

The forest F_t and all learned surfaces are frozen while the nonlinear program is solved. The dependence term changes the marginal cost of different allocations and therefore cannot, in general, be absorbed into the scalar budget b_t.

8.2 Pathwise boundedness and trajectory calibration

The accounting state remains unprojected so that (59) continues to telescope exactly. Boundedness is obtained by saturating the action rather than the state.

Choose finite safety thresholds $q^- < q^+$. Whenever the exact state crosses a threshold, the ordinary controller is bypassed and the following formal extreme actions are imposed:

$$q_t \le q^- \Longrightarrow \mathcal{C}_{t,h} = \mathcal{Y} \text{ for every } h, \qquad q_t \ge q^+ \Longrightarrow \mathcal{C}_{t,1} = \varnothing.$$

Hence trajectories issued in the lower extreme satisfy $Z_t = 0$, whereas those issued in the upper extreme satisfy $Z_t = 1$. These safeguards may lie outside the ordinary box $\mathcal{A}_t$ (for example, outside $[\alpha_{\min}, \alpha_{\max}]^H$); they are boundary conventions used only to establish the pathwise result.

Theorem 3 (Bounded calibration state under right-censored feedback)**.** *Suppose one trajectory is issued per time step, every trajectory label is revealed within at most H steps, and $q_0 \in [q^-, q^+]$. Under the accounting update* (58) *and the extreme-action conventions above,*

$$\boxed{q^- - \eta(1-\beta)H \le q_t \le q^+ + \eta\beta H} \tag{74}$$

for every t, pathwise.

Proof. Consider an excursion above q^+. Every trajectory issued while the state is in that region is forced to have $Z_k = 1$ and therefore contributes $-\eta(1-\beta) \le 0$ when it resolves. The only labels that can subsequently increase the state are successful trajectories issued before the crossing and still unresolved. There are at most H such trajectories, and each contributes at most $\eta\beta$. Hence the upward overshoot is at most $\eta\beta H$.

The lower bound is symmetric. While $q_t \le q^-$, every newly issued trajectory is forced to have $Z_k = 0$ and can only increase the state when it resolves. At most H trajectories issued before the crossing can still resolve as failures, each decreasing the state by at most $\eta(1-\beta)$. Therefore the downward overshoot is at most $\eta(1-\beta)H$. □

Combining Theorem 3 with (61) yields the finite-time bound

$$\left| \frac{1}{T} \sum_{k=1}^{T} (Z_k - \beta) \right| \le \frac{B_q}{\eta T} + \frac{H}{T}, \tag{75}$$

where

$$B_q := \max\left\{ q^+ + \eta\beta H - q_0,\ q_0 - q^- + \eta(1-\beta)H \right\}.$$

For the clipped affine baseline (72), choose

$$q^- = -\frac{\beta}{\kappa}, \qquad q^+ = \frac{1-\beta}{\kappa}.$$

If $q_0 = 0$, the convenient bound

$$\boxed{\left| \frac{1}{T} \sum_{k=1}^{T} Z_k - \beta \right| \le \frac{1}{\kappa\eta T} + \frac{2H}{T}} \tag{76}$$

follows. Thus for fixed H, η, and κ, trajectory-calibration error is $\mathcal{O}(T^{-1})$ pathwise.

8.3 Optimization regret relative to an oracle allocator

Coverage validity and allocation efficiency are evaluated separately. The primary efficiency comparator uses the same realized outer budget sequence b_t and the same predictable forest F_t as the online allocator, but it knows the true pairwise overlap surfaces.

Unlike the convex specialization discussed earlier, we now allow the certified overlap surfaces used by the allocator to be smooth but nonconcave. The resulting fixed-forest allocation problem may therefore be nonconvex. We solve this problem numerically with a feasible, warm-started sequential quadratic programming (SQP) method.

Define the true fixed-forest trajectory-risk function

$$R_t^\star(\boldsymbol{\alpha}) := \sum_{h=1}^{H} \alpha_h - \sum_{e \in F_t} p_{t,e}(\boldsymbol{\alpha}_e), \tag{77}$$

and define the conservative risk function supplied to the SQP solver by

$$\widetilde{R}_t(\boldsymbol{\alpha}) := \sum_{h=1}^{H} \alpha_h - \sum_{e \in F_t} \widetilde{p}_{t,e}(\boldsymbol{\alpha}_e), \tag{78}$$

where

$$\widetilde{p}_{t,e}(\boldsymbol{\alpha}_e) \le p_{t,e}(\boldsymbol{\alpha}_e)$$

on the region queried by the optimizer. No concavity assumption is imposed on $\widetilde{p}_{t,e}$.

The resulting lost overlap credit is

$$\boxed{D_t(\boldsymbol{\alpha}) := \widetilde{R}_t(\boldsymbol{\alpha}) - R_t^\star(\boldsymbol{\alpha}) = \sum_{e \in F_t} [p_{t,e}(\boldsymbol{\alpha}_e) - \widetilde{p}_{t,e}(\boldsymbol{\alpha}_e)] \ge 0.} \tag{79}$$

On an ordinary round, define the clairvoyant true-dependence oracle

$$\boldsymbol{\alpha}_t^\star \in \operatorname*{arg\,max}_{\boldsymbol{\alpha} \in \mathcal{A}_t} \{U_t(\boldsymbol{\alpha}) : R_t^\star(\boldsymbol{\alpha}) \le b_t\}. \tag{80}$$

Thus the oracle receives exactly the same trajectory-risk resource b_t and uses exactly the same forest F_t, but it knows the true pairwise overlap functions.

On extreme-action rounds, define the oracle comparator to use the same forced boundary action, so that the efficiency regret on those rounds is zero by convention.

For the conservative nonlinear problem, define the global value function

$$\widetilde{V}_t(b) := \max_{\boldsymbol{\alpha} \in \mathcal{A}_t} \left\{ U_t(\boldsymbol{\alpha}) : \widetilde{R}_t(\boldsymbol{\alpha}) \le b \right\}. \tag{81}$$

The maximum is well defined, for example, when $\mathcal{A}_t$ is compact and U_t and $\widetilde{R}_t$ are continuous.

Let $\boldsymbol{\alpha}_t$ denote the feasible point returned by SQP, so that

$$\widetilde{R}_t(\boldsymbol{\alpha}_t) \le b_t. \tag{82}$$

Define its global certified optimization gap by

$$\boxed{\epsilon_t^{\mathrm{SQP}} := \widetilde{V}_t(b_t) - U_t(\boldsymbol{\alpha}_t) \ge 0.} \tag{83}$$

The quantity $\epsilon_t^{\mathrm{SQP}}$ measures only numerical/nonconvex optimization error. It is distinct from D_t, which measures the loss caused by imperfect knowledge of the true dependence structure.

Budget sensitivity. Instead of invoking convex strong duality, assume that the certified value function is uniformly Lipschitz with respect to the risk resource over the range of perturbations generated by the dependence error: there exists $\Lambda < \infty$ such that

$$\widetilde{V}_t(b_t + \Delta) - \widetilde{V}_t(b_t) \le \Lambda\Delta \qquad \text{for all } 0 \le \Delta \le \Delta_{\max}. \tag{84}$$

We assume

$$D_t(\boldsymbol{\alpha}_t^\star) \le \Delta_{\max}.$$

The constant Λ again has the interpretation of a maximal marginal value of trajectory-risk budget. Under standard regularity conditions for a parametric nonlinear program, the local derivative of the value function with respect to the active risk constraint is the corresponding Lagrange multiplier. Condition (84), however, is the only property needed below; global convexity and strong duality are not required.

Theorem 4 (Oracle allocation regret with SQP)**.** *Suppose that, on every ordinary round,*

1. *the SQP output is feasible as in* (82)*;*
2. *the overlap surface is conservative, $\widetilde{p}_{t,e} \le p_{t,e}$, on all queried points; and*
3. *the value-function sensitivity bound* (84) *holds.*

Then

$$\boxed{U_t(\boldsymbol{\alpha}_t^\star) - U_t(\boldsymbol{\alpha}_t) \le \epsilon_t^{\mathrm{SQP}} + \Lambda D_t(\boldsymbol{\alpha}_t^\star).} \tag{85}$$

Consequently,

$$\boxed{\mathrm{Reg}_T^{\mathrm{opt}} \le \mathrm{Reg}_T^{\mathrm{SQP}} + \Lambda \sum_{t=1}^{T} D_t(\boldsymbol{\alpha}_t^\star),} \tag{86}$$

where

$$\mathrm{Reg}_T^{\mathrm{SQP}} := \sum_{t=1}^{T} \epsilon_t^{\mathrm{SQP}}. \tag{87}$$

Proof. Since $\boldsymbol{\alpha}_t^\star$ is feasible for the true oracle problem,

$$R_t^\star(\boldsymbol{\alpha}_t^\star) \le b_t.$$

By definition of D_t,

$$\widetilde{R}_t(\boldsymbol{\alpha}_t^\star) = R_t^\star(\boldsymbol{\alpha}_t^\star) + D_t(\boldsymbol{\alpha}_t^\star) \le b_t + D_t(\boldsymbol{\alpha}_t^\star).$$

Hence

$$U_t(\boldsymbol{\alpha}_t^\star) \le \widetilde{V}_t\left(b_t + D_t(\boldsymbol{\alpha}_t^\star)\right).$$

Applying (84) gives

$$U_t(\boldsymbol{\alpha}_t^\star) \le \widetilde{V}_t(b_t) + \Lambda D_t(\boldsymbol{\alpha}_t^\star).$$

Finally, by definition of $\epsilon_t^{\mathrm{SQP}}$,

$$\widetilde{V}_t(b_t) = U_t(\boldsymbol{\alpha}_t) + \epsilon_t^{\mathrm{SQP}}.$$

Substitution proves (85); summing over rounds proves (86). □

Relating the SQP gap to the stopping tolerance. Theorem 4 is valid for any feasible SQP output and does not require that a KKT point be globally optimal. To obtain an explicit solver-regret rate, we impose an additional local tracking condition.

Let

$$r_t^{\mathrm{KKT}}$$

denote the primal-dual KKT residual of the SQP solution. Suppose that the high-coverage operating region contains a regular globally optimal branch of the certified problem and that warm-started SQP remains in its attraction region. Assume there exists a constant $C_{\mathrm{SQP}} < \infty$ such that, throughout this region,

$$\boxed{\epsilon_t^{\mathrm{SQP}} \le C_{\mathrm{SQP}} r_t^{\mathrm{KKT}}.} \tag{88}$$

This type of bound is consistent with the usual local error-bound picture under smoothness, constraint qualification, second-order sufficiency, and an isolated regular optimum.

If SQP is terminated when

$$r_t^{\mathrm{KKT}} \le \tau_t, \tag{89}$$

then

$$\boxed{\mathrm{Reg}_T^{\mathrm{SQP}} \le C_{\mathrm{SQP}} \sum_{t=1}^{T} \tau_t.} \tag{90}$$

Therefore

$$\boxed{\mathrm{Reg}_T^{\mathrm{opt}} \le C_{\mathrm{SQP}} \sum_{t=1}^{T} \tau_t + \Lambda \sum_{t=1}^{T} D_t(\boldsymbol{\alpha}_t^\star).} \tag{91}$$

For example, choosing

$$\tau_t = \frac{1}{t}$$

gives

$$\sum_{t=1}^{T} \tau_t \le 1 + \log T, \tag{92}$$

and therefore

$$\boxed{\mathrm{Reg}_T^{\mathrm{SQP}} \le C_{\mathrm{SQP}}(1 + \log T).} \tag{93}$$

Remark 1 (Local optimality versus global regret)**.** *Because the certified allocation problem may be nonconvex, a small KKT residual alone does not imply a small global optimization gap. The exact regret statement is therefore* (86)*, which retains* $\epsilon_t^{\mathrm{SQP}}$ *explicitly. The stronger residual-based bound* (91) *additionally assumes that warm-started SQP tracks the relevant globally optimal regular branch, or equivalently that the local error bound* (88) *holds relative to the global certified solution. If this cannot be guaranteed analytically, the quantities* $\epsilon_t^{\mathrm{SQP}}$ *can be benchmarked empirically against a multi-start or global/discrete solver on representative rounds.*

Why SQP does not affect coverage validity. Global optimality is not required for the trajectory-risk certificate. Coverage requires only that the issued allocation satisfy the conservative constraint

$$\widetilde{R}_t(\boldsymbol{\alpha}_t) \le b_t.$$

Thus SQP suboptimality affects efficiency through $\epsilon_t^{\mathrm{SQP}}$, but not the validity of the outer trajectory-level calibration argument. In implementation, the final SQP iterate is therefore explicitly checked for feasibility before prediction sets are issued.

Corollary 2 (Stable-dependence regret with SQP)**.** *Suppose H is fixed and, on the simultaneous dependence-certification event, the conservative smooth overlap surfaces satisfy*

$$\sup_{\boldsymbol{\alpha}_e} \left[p_{t,e}(\boldsymbol{\alpha}_e) - \widetilde{p}_{t,e}(\boldsymbol{\alpha}_e) \right] \le c\sqrt{\frac{d_e L_t}{t}} + a_{t,e}, \tag{94}$$

for every $e \in F_t$, where L_t is nondecreasing, $d_e \le H$, and $a_{t,e} \ge 0$ collects any remaining grid, interpolation, smoothing, or other surface-approximation error. Define

$$A_t := \sum_{e \in F_t} a_{t,e}. \tag{95}$$

Since a forest on H vertices contains at most $H-1$ edges,

$$D_t(\boldsymbol{\alpha}_t^\star) \le c(H-1)\sqrt{\frac{H L_t}{t}} + A_t. \tag{96}$$

Consequently, if the SQP residual satisfies $r_t^{\mathrm{KKT}} \le \tau_t$, then

$$\boxed{\begin{aligned} \mathrm{Reg}_T^{\mathrm{opt}} \le\ & C_{\mathrm{SQP}} \sum_{t=1}^{T} \tau_t \\ & + 2\Lambda c(H-1)\sqrt{H L_T T} + \Lambda \sum_{t=1}^{T} A_t + C_{\mathrm{init}}(H), \end{aligned}} \tag{97}$$

where $C_{\mathrm{init}}(H)$ collects any finite burn-in contribution.

In particular, if

$$\tau_t = \frac{1}{t},$$

then

$$\boxed{\begin{aligned} \mathrm{Reg}_T^{\mathrm{opt}} \le\ & C_{\mathrm{SQP}}(1 + \log T) \\ & + 2\Lambda c(H-1)\sqrt{H L_T T} + \Lambda \sum_{t=1}^{T} A_t + C_{\mathrm{init}}(H). \end{aligned}} \tag{98}$$

Hence, if H is fixed,

$$\frac{L_T}{T} \longrightarrow 0, \qquad \frac{1}{T}\sum_{t=1}^{T} A_t \longrightarrow 0,$$

and

$$\frac{1}{T}\sum_{t=1}^{T} \tau_t \longrightarrow 0,$$

then

$$\boxed{\frac{\mathrm{Reg}_T^{\mathrm{opt}}}{T} \longrightarrow 0.} \tag{99}$$

Proof. From (94),

$$D_t(\boldsymbol{\alpha}_t^\star) = \sum_{e \in F_t} \left[p_{t,e}(\boldsymbol{\alpha}_{t,e}^\star) - \widetilde{p}_{t,e}(\boldsymbol{\alpha}_{t,e}^\star) \right].$$

Because $|F_t| \le H-1$, $d_e \le H$, and $L_t \le L_T$,

$$D_t(\boldsymbol{\alpha}_t^\star) \le c(H-1)\sqrt{\frac{HL_t}{t}} + A_t.$$

Using

$$\sum_{t=1}^{T} \frac{1}{\sqrt{t}} \le 2\sqrt{T}$$

gives

$$\sum_{t=1}^{T} D_t(\boldsymbol{\alpha}_t^\star) \le 2c(H-1)\sqrt{HL_T T} + \sum_{t=1}^{T} A_t + C_{\text{init}}(H).$$

Substituting this inequality into (91) proves (97). The choice $\tau_t = 1/t$ and the harmonic-sum bound

$$\sum_{t=1}^{T} \frac{1}{t} \le 1 + \log T$$

give (98). Dividing by T gives the final statement. □

8.4 Two headline guarantees

The framework is therefore summarized by two quantities.

First, under the right-censored accounting update and the extreme-action safeguards, for fixed H,

$$\boxed{\left|\frac{1}{T}\sum_{k=1}^{T}(Z_k - \beta)\right| \le \frac{B_q}{\eta T} + \frac{H}{T},} \tag{100}$$

and hence the realized trajectory-level calibration error converges to zero pathwise.

Second, on the simultaneous dependence-certification event, the efficiency regret satisfies

$$\boxed{\text{Reg}_T^{\text{opt}} \le \text{Reg}_T^{\text{SQP}} + \Lambda \sum_{t=1}^{T} D_t(\boldsymbol{\alpha}_t^\star).} \tag{101}$$

Under the local SQP tracking condition (88),

$$\boxed{\text{Reg}_T^{\text{opt}} \le C_{\text{SQP}} \sum_{t=1}^{T} \tau_t + \Lambda \sum_{t=1}^{T} D_t(\boldsymbol{\alpha}_t^\star).} \tag{102}$$

Thus the efficiency loss separates into two interpretable contributions:

$$\boxed{\text{allocation regret} = \text{SQP optimization error} + \text{value of lost dependence credit.}}$$

For $\tau_t = 1/t$, the cumulative SQP contribution is at most logarithmic, while under stable dependence the statistical dependence-learning contribution is sublinear under the conditions of Corollary 2.

All statistical, numerical, forest-selection, grid, interpolation, and surface-approximation errors remain contributors to the two headline quantities rather than separate performance criteria.

References

[1] D. Hunter. An upper bound for the probability of a union. *Journal of Applied Probability*, 13(3):597–603, 1976. Source.

[2] I. Gibbs and E. J. Candès. Adaptive conformal inference under distribution shift. In *Advances in Neural Information Processing Systems*, 34:1660–1672, 2021. Source.

[3] I. Gibbs and E. J. Candès. Conformal inference for online prediction with arbitrary distribution shifts. *Journal of Machine Learning Research*, 25(162):1–36, 2024. Source.

[4] A. N. Angelopoulos, E. J. Candès, and R. J. Tibshirani. Conformal PID control for time series prediction. In *Advances in Neural Information Processing Systems*, 36, 2023. Source.

A Implementation Specification for a JAX Repository

This appendix specifies the intended software architecture of the method. It is deliberately prescriptive: an implementation agent should be able to construct the repository from this section without inferring algorithmic interfaces from the surrounding theory.

The implementation should be written primarily in JAX. Python objects are used for configuration, orchestration, logging, and discrete choices such as factory construction. Numerical state updates, estimators, controller updates, utility-weight updates, and optimization kernels should use `jax.numpy` and pure functions whenever practical.

The implementation should preserve the following separation:

	trajectory history	
history / grid	↓	efficiency history
↓	feedback signal	↓
dependence estimator	↓	utility weights
	outer controller	

followed by

$$\boxed{(b_t, \boldsymbol{w}_t, \widetilde{R}_t) \longrightarrow \text{SQP/RTI} \longrightarrow \boldsymbol{\alpha}_t.}$$

The feedback estimator and the outer controller must remain separate. For example, a completed-window trajectory-failure signal should be usable without modification by ACI, a PID controller, or any future outer control law.

A.1 Repository layout

The recommended package organization is

```
src/trajectory_risk/
    __init__.py
    configs.py
    history.py
    dependence.py
    signals.py
    controllers.py
```

```
    utility_weights.py
    solver.py
    runner.py

configs/
    default.yaml
    full_history_aci.yaml
    fixed_window_aci.yaml
    adaptive_window_aci.yaml
    adaptive_window_pid.yaml

tests/
    test_history.py
    test_dependence.py
    test_signals.py
    test_controllers.py
    test_utility_weights.py
    test_solver.py
    test_end_to_end.py
```

The numerical modules should not depend on experiment-specific data-loading code. A downstream forecasting project should need to provide only forecast geometry, observed outcomes, critical conformal levels, optional covariates, and prediction-set size information.

A.2 JAX implementation conventions

All stateful numerical components should be implemented in a functional style. A component may expose a Python class for readability, but its numerical state should be passed explicitly:

$$(\text{state}_{t+1}, \text{output}_t) = F(\text{state}_t, \text{input}_t).$$

Abstract base classes are used to select algorithms at construction time. Python polymorphism should *not* occur inside a JIT-compiled function. Instead, the factory chooses a concrete implementation before compilation, and each concrete implementation calls fixed-shape JAX kernels.

The following rules should be enforced.

1. Numerical kernels use `jax.numpy` rather than `numpy`. NumPy is permitted only for file I/O, plotting, testing, and conversion at the repository boundary.

2. Runtime numerical states should be immutable PyTrees. Standard dataclasses registered with `jax.tree_util.register_pytree_node_class` are sufficient.

3. Array shapes should be static whenever possible. Online histories therefore use fixed-capacity ring buffers and masks rather than Python lists inside JIT-compiled code.

4. Data-dependent branching inside compiled kernels should use `jax.lax.cond`; repeated updates should use `jax.lax.scan` or `jax.lax.fori_loop`; independent horizon or edge operations should use `jax.vmap`.

5. Configuration objects are ordinary Python dataclasses parsed by Pyrallis. They are not required to be PyTrees because configuration parsing occurs before the numerical loop is compiled.

6. The implementation should support both `float32` and `float64`. Numerical validation and regression tests should use `float64`; GPU benchmarking may additionally use `float32`.

7. No estimator may use an observation before its mathematically defined reveal time. Tests should explicitly check this no-lookahead property.

A.3 `history.py`: common grid and trajectory history

The file `history.py` is the common data layer. It should own the marginal-risk grid, all numerical trajectory records required by the remaining modules, and utility statistics derived from prediction-set geometry.

It should *not* contain the dependence-learning rule, feedback-control rule, or SQP algorithm.

Risk grid. For

$$\mathcal{G} = \{a_1, \dots, a_M\},$$

store

$$\boldsymbol{a} = (a_1, \dots, a_M) \in \mathbb{R}^M.$$

The bivariate query grid is generated by broadcasting rather than stored as a Python list:

$$\mathcal{Q} = \mathcal{G}^2.$$

The grid object should validate

$$0 < a_1 < \cdots < a_M < 1.$$

A useful public data structure is conceptually

```
@dataclass(frozen=True)
class AlphaGrid:
    values: jax.Array          # shape (M,)
```

The grid size M is static for a compiled experiment.

Trajectory ring buffer. The history should retain at least

$$H + \max\{W^{\mathrm{dep}}_{\max}, W^{\mathrm{sig}}, W^{\mathrm{util}}\}$$

origins, where the three window lengths correspond to dependence estimation, trajectory-signal estimation, and utility estimation.

A trajectory record should include numerical arrays equivalent to

```
origin_id              shape (C,)
issued_alpha           shape (C,H)
critical_level         shape (C,H)
critical_resolved      shape (C,H)
trajectory_label       shape (C,)
trajectory_resolved    shape (C,)
first_failure          shape (C,)
reveal_time            shape (C,)
set_size_grid          shape (C,H,M)   # optional
valid_origin           shape (C,)
```

where C is the ring-buffer capacity.

Use the following coding conventions:

`trajectory_label` $= -1$	means unresolved,
`trajectory_label` $= 0$	means successful trajectory,
`trajectory_label` $= 1$	means failed trajectory,
`first_failure` $= -1$	means unresolved,
`first_failure` $= h$	means first failure at horizon h,
`first_failure` $= H$	means complete success.

The public API should provide pure update functions corresponding to

```
add_origin(...)
resolve_horizon(...)
resolve_trajectory(...)
prune_or_overwrite(...)
get_recent_completed_origins(...)
get_edge_critical_levels(...)
get_first_failure_histogram(...)
get_set_size_curve(...)
get_marginal_set_efficiency(...)
```

Actual Python prediction-set objects should not be stored inside the JAX state. If the underlying prediction set is geometrically complicated, the repository boundary should convert it into numerical quantities needed by the algorithm, such as its critical conformal level and its size evaluated on $\mathcal{G}$.

Critical-level event surfaces. Once the critical levels

$$A^\star_{k,s}, \qquad A^\star_{k,r}$$

for edge $e = (s, r)$ are known, construct the full lower-orthant indicator surface by broadcasting:

$$X_{k,e}(a_i, a_j) = \mathbf{1}\{A^\star_{k,s} < a_i\}\mathbf{1}\{A^\star_{k,r} < a_j\}.$$

The implementation should construct this surface with a pair of vectorized comparisons and a logical conjunction. No Python loop over the M^2 grid points is required.

Set-size statistics. If

$$S_{k,h}(a_m)$$

is available, store the full size curve on $\mathcal{G}$. A centered finite difference supplies the marginal efficiency

$$\widehat{m}_{k,h}(a_m) = -\frac{S_{k,h}(a_{m+1}) - S_{k,h}(a_{m-1})}{a_{m+1} - a_{m-1}}.$$

Endpoint differences should use one-sided formulas.

This statistic is exposed to `utility_weights.py`; it should not be used to modify the coverage-accounting state.

A.4 `dependence.py`: pairwise overlap estimation

The dependence module estimates

$$p_{t,e}(a,b) = \Pr(V_{t,s}(a) = 1,\, V_{t,r}(b) = 1 \mid \mathcal{F}_t), \qquad e = (s,r).$$

Although one may colloquially call this the "correlation estimator," the implementation should use the name *dependence estimator*, because the object being estimated is a bivariate lower-orthant probability surface rather than a Pearson correlation coefficient.

All estimators inherit from a common abstract interface.

```
class PairwiseDependenceEstimator(ABC):

    @abstractmethod
    def init_state(self, history, edges, grid):
        ...

    @abstractmethod
    def update(self, state, history, t):
        ...

    @abstractmethod
    def estimate(self, state, history, t):
        """
        Return point estimates and, when valid, certified
        lower overlap surfaces for every candidate edge.
        """
        ...
```

The returned object should contain at least

```
point_surface     # shape (E,M,M)
lower_surface     # shape (E,M,M)
radius            # shape (E,) or (E,M,M)
sample_count      # shape (E,)
certified         # shape (E,) boolean
ready             # shape (E,) boolean
```

where $E = |\mathcal{E}|$.

The implementation should support exactly three initial estimators.

A.4.1 Full-history estimator

The full-history estimator uses every fully resolved origin available to an edge. It should maintain cumulative counts rather than recomputing from the ring buffer.

For each edge,

$$C_{t,e}(g) = \sum_{k \in \mathcal{I}_{t,e}} X_{k,e}(g), \qquad N_{t,e} = |\mathcal{I}_{t,e}|,$$

and

$$\widehat{p}_{t,e}(g) = \frac{C_{t,e}(g)}{N_{t,e}}.$$

Under the stable-dependence certificate used in the main text, the lower surface is

$$\underline{p}_{t,e}(g) = [\widehat{p}_{t,e}(g) - E_{t,e}]_+ ,$$

with

$$E_{t,e} = \sqrt{\frac{d_e}{2N_{t,e}} \log\left(\frac{\pi^2 C_{\mathcal{E}} N_{t,e}^2}{3\delta_{\mathrm{dep}}}\right)},$$

and

$$C_{\mathcal{E}} = M^2 \sum_{e \in \mathcal{E}} d_e.$$

If $N_{t,e} = 0$, set the point estimate to zero, the certified lower surface to zero, and `ready=False`.

A.4.2 Fixed-window estimator

The fixed-window estimator uses the W most recent edge origins that are fully resolved at physical time t:

$$\widehat{p}_{t,e}^{(W)}(g) = \frac{1}{W} \sum_{m=0}^{W-1} X_{t-d_e-m,e}(g),$$

after burn-in.

The implementation should index these origins by their origin identifiers, not by reveal-order position. This preserves the mathematical meaning of the delayed window.

The point estimate is always returned after burn-in. A field `certified=True` may be set only when the configured statistical assumptions justify a one-sided current-time lower confidence surface. Otherwise, the module should return

`certified=False`

rather than silently treating a plug-in estimate as a certificate.

A.4.3 Adaptive-regret window estimator

The adaptive estimator maintains a dictionary

$$1 \le W_1 < \cdots < W_J.$$

Expert j forms

$$\widehat{p}_{t,e,j}(g) = \frac{1}{W_j} \sum_{m=0}^{W_j-1} X_{t-d_e-m,e}(g).$$

For edge e, maintain a separate fixed-share learner for each residue class modulo d_e. A convenient static JAX layout is

$$\texttt{weights} \in \mathbb{R}^{E \times H \times J},$$

together with a Boolean mask identifying which residue slots are valid for each edge.

For an origin k, let

$$c = k \bmod d_e.$$

The prediction made for that origin is

$$\widehat{p}_{k,e}^{\mathrm{mix}} = \sum_{j=1}^{J} \pi_{k,e,j} \widehat{p}_{k,e,j}.$$

Because expert losses are evaluated only after the edge observation is revealed, store the expert predictions made at the origin. Since $d_e \le H$, only a bounded number of predictions are pending. A static array of shape

$$(H+1, E, J, M, M)$$

is sufficient when indexed by origin modulo $H+1$.

When $B_{k,e}$ is revealed, compute

$$\ell_{k,e,j} = \frac{1}{M^2} \sum_{g \in \mathcal{Q}} \left(X_{k,e}(g) - \widehat{p}_{k,e,j}(g) \right)^2 .$$

For each residue stream initialize

$$w_{e,c,j} = 1.$$

Let

$$\sigma_{\mathrm{fs}} = \frac{1}{n_{\max}}, \qquad 0 < \eta_{\mathrm{fs}} \le \frac{1}{2}.$$

The update must implement

$$\widetilde{w}_{e,c,j} = w_{e,c,j} \exp(-\eta_{\mathrm{fs}} \ell_{k,e,j}),$$

followed by

$$\boxed{w^{+}_{e,c,j} = (1 - \sigma_{\mathrm{fs}}) \widetilde{w}_{e,c,j} + \frac{\sigma_{\mathrm{fs}}}{J} \sum_{\ell=1}^{J} \widetilde{w}_{e,c,\ell}.}$$

Normalize before the next prediction:

$$\pi_{e,c,j} = \frac{w_{e,c,j}}{\sum_{\ell=1}^{J} w_{e,c,\ell}}.$$

This is the adaptive-regret estimator analyzed in the companion delayed bivariate note. Codex should reproduce these updates exactly rather than replace them with an arbitrary exponential moving average.

The adaptive estimator should expose both the mixture and every base expert for diagnostics:

```
mixture_surface
expert_surfaces
expert_weights
expert_losses
```

Certification versus plug-in estimation. Point estimation and one-sided certification must be distinct fields in the software API. In particular, the nonstationary adaptive-regret guarantee is an estimation guarantee and does not automatically imply that

$$\widehat{p}_{t,e}^{\mathrm{mix}}$$

is a conservative current-time lower bound.

The solver therefore receives a flag

`require_certified_dependence`.

If this flag is true and the selected estimator cannot provide a certified lower surface, the code should either fall back to zero overlap credit (Boole's inequality) or raise a configuration error. It must not silently use an uncertified plug-in surface as if it were a lower confidence bound.

A.5 `signals.py`: trajectory-feedback construction

The signal module answers only the question

> What provisional value should be assigned to the eventual trajectory label while that trajectory is unresolved?

It does not decide how strongly the budget should react. That is the role of `controllers.py`. Let

$$\widehat{z}_{k,t} \in [0,1]$$

be the provisional value of Z_k at physical time t.

Every signal implementation uses the same predict-and-correct identity. At issuance, trajectory k contributes

$$\beta - \widehat{z}_{k,k}.$$

Whenever its estimate changes,

$$\widehat{z}_{k,t-1} \longrightarrow \widehat{z}_{k,t},$$

it contributes

$$\widehat{z}_{k,t-1} - \widehat{z}_{k,t}.$$

At resolution, set

$$\widehat{z}_{k,\rho_k} = Z_k.$$

The complete contribution telescopes to

$$\beta - Z_k.$$

The abstract interface should therefore emit *signed atomic increments*, not directly update the outer budget.

```
class TrajectorySignal(ABC):

    @abstractmethod
    def init_state(self, history):
        ...

    @abstractmethod
    def observe(self, state, history, t):
        """
        Process newly available outcomes and revisions of
        pending trajectory probabilities.
        Return a fixed-size SignalPacket.
        """
```

```
        ...

    @abstractmethod
    def issue(self, state, history, t, alpha, features):
        """
        Register the newly issued trajectory and return its
        issue-time predict-and-correct increment.
        """
        ...
```

A `SignalPacket` should contain

```
increments          shape (H+1,)
event_mask          shape (H+1,)
resolved_labels     shape (H,)
resolved_mask       shape (H,)
critical_budget     scalar, optional
critical_valid      boolean
```

The fixed event dimension allows controller updates to use `jax.lax.scan`. The sum of all active entries in `increments` is the total predict-and-correct feedback arriving at that call.

The first implementation should contain four signal classes.

A.5.1 Reveal-only signal

For every unresolved trajectory set

$$\widehat{z}_{k,t} = \beta.$$

The issue-time increment is therefore zero. When Z_k resolves, emit

$$\beta - Z_k.$$

This reproduces the original delayed-only controller.

A.5.2 Pending-is-valid signal

For every unresolved trajectory set

$$\widehat{z}_{k,t} = 0.$$

At issuance emit

$$+\beta.$$

If the trajectory later fails, emit the correction

$$-1.$$

If it succeeds, emit no terminal correction.

Thus a failed trajectory contributes

$$\beta - 1,$$

and a successful trajectory contributes

$$\beta,$$

exactly as required.

A.5.3 Completed-window probability signal

For a window W_{sig}, define

$$\widehat{\pi}_t = \frac{1}{W_{\text{sig}}} \sum_{k=t-H-W_{\text{sig}}+1}^{t-H} Z_k.$$

Only fully resolved origin cohorts are used.

At issue time,

$$\widehat{z}_{t,t} = \widehat{\pi}_t.$$

The configuration should contain

`refresh_pending: bool`

If false, each trajectory keeps its issue-time probability until resolution. If true, every still-pending trajectory is updated to the newest $\widehat{\pi}_t$, and the corresponding correction

$$\widehat{z}_{k,t-1} - \widehat{z}_{k,t}$$

is emitted. The latter is the default because it uses newly available information while retaining exact telescoping.

During burn-in, use a configurable prior

$$\pi_0,$$

with default

$$\pi_0 = \beta.$$

A.5.4 Learned soft-probability signal

This implementation predicts

$$\widehat{z}_{k,t} \approx \Pr(Z_k = 1 \mid \text{predictable features}).$$

The initial implementation should provide a small JAX logistic model rather than depend on an external machine-learning package:

$$\widehat{z} = \sigma(\theta^\top x).$$

Update θ only from labels that have actually resolved.

The features may include

$$\boldsymbol{\alpha}_k, \quad h, \quad \text{forecast uncertainty summaries}, \quad \text{recent trajectory statistics},$$

and, for a pending trajectory, its amount of survived prefix.

The learned model is used only for provisional feedback. It is not a coverage certificate, and model misspecification cannot change the telescoping identity because the final estimate is always corrected to Z_k.

A future implementation may replace the direct logistic model with the first-failure hazard model described in the main text without changing the `TrajectorySignal` interface.

A.6 `controllers.py`: reaction to trajectory feedback

The controller module answers

> Given a signed calibration signal, how should the scalar trajectory-risk budget evolve?

Every outer control law inherits from one interface.

```
class OuterBudgetController(ABC):

    @abstractmethod
    def init_state(self, beta):
        ...

    @abstractmethod
    def update(self, state, signal_packet):
        ...

    @abstractmethod
    def propose_budget(self, state):
        ...
```

The controller should return a proposal

$$\widetilde{b}_t \in [0, 1].$$

The exact accounting safeguard described below is applied afterward.

ACI controller. The fixed-gain implementation maintains a scalar budget state. For every atomic signal increment g,

$$\boxed{b^+ = \Pi_{[0,1]}\left(b + \gamma g\right).}$$

All active events in a `SignalPacket` are processed sequentially with `jax.lax.scan`.

The sign convention must be tested explicitly:

$$Z = 1 \quad \Rightarrow \quad g = \beta - 1 < 0 \quad \Rightarrow \quad b \text{ decreases},$$

so prediction sets become more conservative after excess failures.

DtACI controller. A literal implementation of DtACI should follow Gibbs and Candès, *Conformal Inference for Online Prediction with Arbitrary Distribution Shifts*, and the authors' repository `isgibbs/DtACI`.

Maintain candidate gains

$$\gamma_1, \ldots, \gamma_J$$

and one ACI state per candidate.

The implementation must distinguish literal DtACI from a generic adaptive-gain ensemble. Literal DtACI requires the scalar nestedness object used by its pinball-loss update. In the present notation this is a critical trajectory budget $B_k^\star$ satisfying

$$Z_k(b) = \mathbf{1}\{B_k^\star < b\}.$$

Therefore `DtACIController` should declare

`requires_critical_budget = True`

and raise a configuration error if the chosen signal/problem representation does not provide one.

If scalar nestedness has not been established, a separate class such as `FixedShareGainController` may aggregate several ACI gains using a chosen online loss, but it should not be labeled as literal DtACI in plots or theory.

PID and Conformal-PID controllers. The controller interface should also contain a generic `PIDBudgetController`. For a scalar control error e_t, a basic form is

$$u_t = K_P e_t + K_I I_t + K_D(e_t - e_{t-1}),$$

with

$$I_t = I_{t-1} + e_t.$$

Then

$$\widetilde{b}_{t+1} = \Pi_{[0,1]}(b_0 + u_t).$$

The integral state should support optional saturation or anti-windup.

For a faithful implementation of Conformal PID, Codex should read Angelopoulos, Candès, and Tibshirani, *Conformal PID Control for Time Series Prediction*, together with the authors' repository `aangelopoulos/conformal-time-series`, in particular the control routines in `core/methods.py`. The repository contains the quantile tracker, integral correction, saturation functions, and optional scorecasting logic.

If those equations are modified to consume the generic predict-and-correct signal of this paper rather than the conformal-score input of the original method, the implementation should be named `PIDBudgetController` or `ConformalPIDStyleController`, not presented as an unchanged implementation of the original algorithm.

Exact accounting safeguard. The model-free accounting state remains separate from every controller above:

$$q_t = q_{t-1} + \eta_q \sum_{k \in \mathcal{R}_t} (\beta - Z_k).$$

Implement this as a small class or pure function in `controllers.py`, for example `ExactAccountingGuard`.

Given the ordinary controller proposal $\widetilde{b}_t$, return

$$\boxed{b_t = \begin{cases} 0, & q_t \le q^-, \\ 1, & q_t \ge q^+, \\ \widetilde{b}_t, & q^- < q_t < q^+. \end{cases}}$$

The signal estimator may be wrong and the ordinary controller may be suboptimal; neither is allowed to bypass this safety layer.

A.7 `utility_weights.py`: modular utility adaptation

The utility-weight module controls

$$\boldsymbol{w}_t = (w_{t,1}, \ldots, w_{t,H})$$

in

$$U_t(\boldsymbol{\alpha}) = \sum_{h=1}^{H} w_{t,h} \log \alpha_h.$$

It should inherit from

```
class UtilityWeightUpdater(ABC):

    @abstractmethod
    def init_state(self, horizon):
        ...

    @abstractmethod
    def update(self, state, history, alpha_prev, t):
        ...

    @abstractmethod
    def weights(self, state):
        ...
```

At least the following implementations should be provided.

Static weights. Return a configured vector

$$\boldsymbol{w}^{(0)}, \qquad w_h^{(0)} > 0, \qquad \sum_h w_h^{(0)} = 1.$$

Marginal-efficiency weights. Using the set-size statistics from `history.py`, estimate

$$\widehat{m}_{t,h} \approx -\frac{\partial S_{t,h}}{\partial \alpha_h}.$$

Form

$$\widetilde{w}_{t,h}^{\mathrm{tar}} = c_h \alpha_{t-1,h} \widehat{m}_{t,h}.$$

Failure-aware efficiency weights. Estimate the recent first-failure mass

$$\widehat{f}_{t,h} = \frac{1}{W_{\mathrm{util}}} \sum_{k=t-H-W_{\mathrm{util}}+1}^{t-H} \mathbf{1}\{\tau_k = h\}.$$

Then form

$$\widetilde{w}_{t,h}^{\mathrm{tar}} = \frac{c_h \alpha_{t-1,h} \widehat{m}_{t,h} + \varepsilon_m}{\varepsilon_f + \widehat{f}_{t,h}}.$$

Normalize and enforce

$$w_{\min} \le w_{t,h} \le w_{\max}, \qquad \sum_h w_{t,h} = 1.$$

A convenient bounded-simplex projection uses a scalar dual variable:

$$w_h = \mathrm{clip}(v_h - \nu, w_{\min}, w_{\max}),$$

where ν is found by fixed-iteration bisection so that

$$\sum_h w_h = 1.$$

The bisection should use `jax.lax.fori_loop`.

To make the update RTI-compatible, first project the raw target into the admissible weight set, then apply

$$\boldsymbol{w}_{t+1}^{\mathrm{ema}} = (1-\zeta_w)\boldsymbol{w}_t + \zeta_w \boldsymbol{w}_t^{\mathrm{tar}}.$$

Finally enforce

$$\|\boldsymbol{w}_{t+1} - \boldsymbol{w}_t\|_2 \le \delta_w$$

by scaling the proposed difference:

$$\boldsymbol{d}_t = \boldsymbol{w}_{t+1}^{\mathrm{ema}} - \boldsymbol{w}_t,$$

$$\boldsymbol{w}_{t+1} = \boldsymbol{w}_t + \min\left\{1, \frac{\delta_w}{\|\boldsymbol{d}_t\|_2 + \varepsilon}\right\} \boldsymbol{d}_t.$$

Because this is a convex combination of two admissible weight vectors, it preserves the simplex and box constraints.

All weight updates must use only information available before the round-t SQP problem is solved.

A.8 `solver.py`: JAX SQP and real-time iteration

The inner nonlinear program is

$$\min_{\boldsymbol{\alpha}} f_t(\boldsymbol{\alpha}) := -\sum_{h=1}^{H} w_{t,h} \log \alpha_h$$

subject to

$$c_t(\boldsymbol{\alpha}) := \widetilde{R}_t(\boldsymbol{\alpha}) - b_t \le 0,$$

and

$$\alpha_{\min} \le \alpha_h \le \alpha_{\max}.$$

The solver should be implemented in JAX and should support both a full SQP mode and a one-step real-time iteration mode.

Reference implementation to study. Codex should first inspect the Trustworthy Robotics Lab repository `trustworthyrobotics/gpu_sls`, especially

```
src/gpu_sls/gpu_sqp.py
src/gpu_sls/gpu_admm.py
```

and the associated Fang–Chou GPU-SLS paper.

The implementation should borrow the following software patterns:

1. immutable SQP and QP configuration dataclasses registered as JAX PyTrees;

2. a JIT-compiled search-direction routine;

3. warm starts for primal variables, dual variables, and the QP solver;

4. fixed-shape iteration through `jax.lax.fori_loop`;

5. convergence checks based on constraint feasibility and relative step size;

6. a merit function and Armijo-style backtracking line search;

7. retention of QP state across consecutive real-time iterations.

The SLS-specific dynamics, obstacle constraints, reachable-set variables, and system-level-synthesis routines should *not* be copied into this repository.

If source code is copied rather than independently reimplemented, preserve the source repository's MIT license notice and attribution.

SQP subproblem. At iterate $\boldsymbol{\alpha}$, form

$$g_f = \nabla f_t(\boldsymbol{\alpha}), \qquad g_c = \nabla c_t(\boldsymbol{\alpha}).$$

The quadratic subproblem is

$$\begin{aligned} \min_{\boldsymbol{d}} \quad & \frac{1}{2}\boldsymbol{d}^\top B \boldsymbol{d} + g_f^\top \boldsymbol{d} \\ \text{s.t.} \quad & c_t(\boldsymbol{\alpha}) + g_c^\top \boldsymbol{d} \le 0, \\ & \alpha_{\min} - \boldsymbol{\alpha} \le \boldsymbol{d} \le \alpha_{\max} - \boldsymbol{\alpha}. \end{aligned} \tag{103}$$

The initial robust implementation should use

$$B = \nabla^2 f_t(\boldsymbol{\alpha}) + \mu I, \qquad \mu > 0,$$

which is positive definite for the weighted log utility. A later option may use a regularized Lagrangian Hessian.

All derivatives should be obtained from `jax.grad`, `jax.jacfwd`, or `jax.hessian`; hand-coded derivatives should be used only after profiling shows a need.

QP backend. Two backends are useful.

The primary backend may adapt the GPU-SLS ADMM pattern to this much smaller dense QP. Because the dimension is only H, the implementation should remove all trajectory-dynamics structure and retain only the generic warm-started ADMM/QP logic.

A second backend may use JAXopt's `BoxOSQP` or `OSQP` as a reference implementation for testing. These solvers are JAX-compatible and support JIT compilation, PyTrees, and warm starts. The repository should not depend on CVXPY in the online execution path.

Line search and feasibility filter. A useful merit function is

$$\mathcal{M}(\boldsymbol{\alpha}) = f_t(\boldsymbol{\alpha}) + \rho_{\text{merit}}[c_t(\boldsymbol{\alpha})]_+.$$

Full SQP mode should perform Armijo backtracking on the QP direction.

RTI mode may use only one SQP step, but the issued allocation must still pass an explicit feasibility check:

$$\widetilde{R}_t(\boldsymbol{\alpha}_t) \le b_t + \varepsilon_{\text{feas}}.$$

If the proposed step is infeasible, backtrack along

$$\boldsymbol{\alpha}_{\text{old}} + \xi \boldsymbol{d}, \qquad \xi \in (0, 1],$$

until feasibility is recovered.

If no acceptable point is found, interpolate toward a known conservative feasible allocation, for example the configured $\boldsymbol{\alpha}_{\min}$. If the conservative fallback is itself infeasible due to a numerical or modeling error, the solver must return a failure flag and the outer system must issue the conservative extreme prediction-set action.

Numerical optimization failure must never silently bypass the risk constraint.

Warm-start state. Store at least

```
alpha
risk_dual
box_duals
qp_primal
qp_dual
qp_penalty
last_kkt_residual
last_feasibility
```

for use on the next round.

Changing $\boldsymbol{w}_t$ should require only recomputing the objective derivatives. Changing b_t shifts the scalar risk constraint. Updating the learned dependence surface changes the risk function and its derivatives.

These slowly varying inputs make the problem suitable for RTI.

Public solver interface. A useful interface is

```
@dataclass(frozen=True)
class SolverResult:
    alpha: jax.Array
    risk_dual: jax.Array
    objective: jax.Array
    risk_value: jax.Array
    kkt_residual: jax.Array
    feasible: jax.Array
    converged: jax.Array
    iterations: jax.Array

class SQPSolver:
    def init_state(...):
        ...

    def solve(
        self,
        state,
        budget,
```

```
        weights,
        risk_model,
    ) -> tuple[SolverState, SolverResult]:
        ...
```

The `risk_model` passed to the compiled kernel should resolve to a fixed JAX callable before JIT compilation.

A.9 `configs.py`: Pyrallis configuration

Configuration should use nested Python dataclasses and Pyrallis. The relevant reference implementation is the repository `eladrich/pyrallis`.

Use one dataclass per module and one top-level configuration:

```
@dataclass
class JaxConfig:
    enable_x64: bool = True
    platform: str = "auto"

@dataclass
class HistoryConfig:
    horizon: int = 10
    alpha_min: float = 1e-3
    alpha_max: float = 0.25
    grid_size: int = 51
    capacity: int = 512

@dataclass
class DependenceConfig:
    kind: str = "adaptive"
    delta: float = 0.05
    window: int = 100
    windows: tuple[int, ...] = (16, 32, 64, 128)
    meta_eta: float = 0.1
    n_max: int = 100000
    require_certificate: bool = True

@dataclass
class SignalConfig:
    kind: str = "completed_window"
    window: int = 100
    refresh_pending: bool = True
    prior_probability: float | None = None
    learning_rate: float = 1e-2

@dataclass
class ControllerConfig:
    kind: str = "aci"
```

```
    budget_init: float = 0.1
    gamma: float = 0.01
    dtaci_gammas: tuple[float, ...] = (...)
    kp: float = 0.01
    ki: float = 0.0
    kd: float = 0.0

@dataclass
class WeightConfig:
    kind: str = "failure_aware"
    smoothing: float = 0.05
    max_step_norm: float = 0.05
    min_weight: float = 0.01
    max_weight: float = 0.8
    failure_window: int = 100
    eps_failure: float = 1e-3
    eps_efficiency: float = 1e-6

@dataclass
class SolverConfig:
    mode: str = "rti"
    qp_backend: str = "admm"
    max_sqp_iterations: int = 1
    feasibility_tol: float = 1e-6
    step_tol: float = 1e-5
    hessian_regularization: float = 1e-6
    line_search: bool = True
    warm_start: bool = True

@dataclass
class SafetyConfig:
    eta_q: float = 0.01
    q_lower: float = -1.0
    q_upper: float = 1.0

@dataclass
class ExperimentConfig:
    beta: float = 0.1
    jax: JaxConfig = field(default_factory=JaxConfig)
    history: HistoryConfig = field(default_factory=HistoryConfig)
    dependence: DependenceConfig = field(
        default_factory=DependenceConfig)
    signal: SignalConfig = field(default_factory=SignalConfig)
    controller: ControllerConfig = field(
        default_factory=ControllerConfig)
    weights: WeightConfig = field(default_factory=WeightConfig)
    solver: SolverConfig = field(default_factory=SolverConfig)
    safety: SafetyConfig = field(default_factory=SafetyConfig)
```

Factories should map the string-valued `kind` field to concrete ABC implementations. Invalid combinations should fail at startup.

For example, requesting literal DtACI without a critical trajectory budget should raise an error before the experiment begins.

Example YAML. A representative configuration is

```
beta: 0.10

jax:
  enable_x64: true
  platform: auto

history:
  horizon: 10
  alpha_min: 0.001
  alpha_max: 0.25
  grid_size: 51
  capacity: 512

dependence:
  kind: adaptive
  delta: 0.05
  window: 100
  windows: [16, 32, 64, 128]
  meta_eta: 0.10
  n_max: 100000
  require_certificate: true

signal:
  kind: completed_window
  window: 100
  refresh_pending: true
  prior_probability: 0.10

controller:
  kind: aci
  budget_init: 0.10
  gamma: 0.01
  kp: 0.01
  ki: 0.00
  kd: 0.00

weights:
  kind: failure_aware
  smoothing: 0.05
  max_step_norm: 0.05
  min_weight: 0.01
```

```
  max_weight: 0.80
  failure_window: 100
  eps_failure: 0.001
  eps_efficiency: 0.000001

solver:
  mode: rti
  qp_backend: admm
  max_sqp_iterations: 1
  feasibility_tol: 0.000001
  step_tol: 0.00001
  hessian_regularization: 0.000001
  line_search: true
  warm_start: true

safety:
  eta_q: 0.01
  q_lower: -1.0
  q_upper: 1.0
```

The executable entry point should parse this directly with Pyrallis and allow command-line overrides of nested fields.

JAX platform and precision configuration must be applied before large arrays or compiled functions are created.

A.10 `runner.py`: exact order of operations

The runner is an orchestrator only. It should contain as little mathematics as possible.

At physical time t, execute the following order.

1. Observe Y_t.
2. Update every active horizon whose target resolves at t, compute its critical level, and write the result into `history.py`.
3. Determine whether any trajectory labels become resolved. Update the exact accounting state
$$q_t = q_{t-1} + \eta_q \sum_{k \in \mathcal{R}_t} (\beta - Z_k).$$
4. Pass the new information to the selected `TrajectorySignal`. Obtain any correction events caused by newly resolved outcomes or refreshed provisional probabilities.
5. Apply those events to the selected `OuterBudgetController`.
6. Update the selected `PairwiseDependenceEstimator` using all edge observations that have become available.
7. Compute the predictable utility weights $\boldsymbol{w}_t$ from `utility_weights.py`.
8. Ask the controller for its ordinary proposal $\widetilde{b}_t$.

9. Apply the exact accounting guard to obtain the issued budget b_t.
10. Freeze the predictable forest, dependence surface, budget, and utility weights for the current solve.
11. Warm-start `solver.py` from the previous primal-dual state and perform either the configured number of SQP iterations or one RTI step.
12. Verify
$$\widetilde{R}_t(\boldsymbol{\alpha}_t) \leq b_t + \varepsilon_{\text{feas}}.$$
If necessary perform the feasibility restoration described above.
13. Construct and issue the H prediction sets corresponding to $\boldsymbol{\alpha}_t$.
14. Store the issued allocation, optional set-size curves, and all numerical metadata needed by future estimators in `history.py`.
15. Call `TrajectorySignal.issue(...)` to obtain the provisional issue-time signal for the newly issued trajectory. Apply that event to the outer controller state so that it affects the next round, not the already-issued allocation at round t.

This ordering prevents circular dependence between the budget, the SQP allocation, and the provisional failure probability assigned to the newly issued trajectory.

A.11 Factories and experiment modularity

The entry point should contain factory functions conceptually equivalent to

```
build_dependence_estimator(cfg.dependence)
build_signal(cfg.signal)
build_controller(cfg.controller)
build_weight_updater(cfg.weights)
build_solver(cfg.solver)
```

No experiment script should contain an `if/elif` chain implementing algorithmic formulas. All algorithm-specific logic belongs in the corresponding module.

Consequently, the experiment

```
adaptive dependence
+ completed-window signal
+ ACI
+ failure-aware utility
+ RTI-SQP
```

can be changed to

```
fixed-window dependence
+ learned probability signal
+ PID
+ static utility
+ full SQP
```

by modifying YAML only.

A.12 Required tests and implementation invariants

The repository should not be considered complete until the following properties are covered by automated tests.

History tests. Ring-buffer indexing must recover the correct origins after wraparound. No critical level may be marked resolved before its physical reveal time. Completed-window queries must contain only fully resolved origin cohorts.

Dependence tests. For a small deterministic example, every estimator must match a brute-force NumPy implementation. The full-history cumulative-count update must equal recomputation from all observations. The fixed-window estimator must use the correct origin indices. The adaptive estimator must reproduce the fixed-share equations exactly, maintain weights summing to one, and update only the residue class associated with the revealed origin.

Signal tests. For every signal type and every simulated trajectory,

$$\sum_{\text{all events generated by trajectory } k} g = \beta - Z_k.$$

This telescoping test is mandatory.

The reveal-only method must emit zero at issuance. The pending-valid method must emit $+\beta$ at issuance and -1 only when a later failure occurs. The completed-window method must never use an unresolved trajectory in $\widehat{\pi}_t$.

Controller tests. A failure event must move an ACI budget in the conservative direction and a success event in the aggressive direction. Every controller proposal must remain inside its configured budget interval. The exact safety guard must override every ordinary controller at the two extreme state boundaries.

Utility-weight tests. Weights must remain positive, sum to one to numerical tolerance, respect configured lower and upper bounds, and satisfy

$$\|\boldsymbol{w}_{t+1} - \boldsymbol{w}_t\|_2 \leq \delta_w.$$

Solver tests. Compare the JAX SQP result on small synthetic problems against a trusted high-accuracy offline nonlinear solver. Test gradients against finite differences. Test the QP backend independently. Test warm and cold starts. Every result with `feasible=True` must satisfy the certified risk constraint numerically.

RTI tests should verify that slowly changing b_t and $\boldsymbol{w}_t$ are tracked from the previous solution without reinitialization.

JAX tests. Every core numerical update should run both normally and under `jax.jit`. Where mathematically appropriate, edge or horizon batches should also run under `jax.vmap`. JIT and non-JIT outputs must agree within tolerance.

End-to-end accounting test. For any synthetic stream, verify numerically that

$$q_T - q_0 = \eta_q \sum_{k \in \mathcal{D}_T} (\beta - Z_k)$$

and independently verify the predict-and-correct telescoping identity for the selected soft signal.

These tests protect the two most important invariants of the repository: no lookahead and exact trajectory-level accounting.

A.13 External implementation references for Codex

Before implementing the corresponding components, Codex should inspect the following public sources.

JAX SQP and GPU optimization. Trustworthy Robotics Lab, GitHub repository `trustworthyrobotics/gpu_` especially `src/gpu_sls/gpu_sqp.py` and `src/gpu_sls/gpu_admm.py`. The associated paper is Fang and Chou, *Safe Large-Scale Robust Nonlinear MPC in Milliseconds via Reachability-Constrained System Level Synthesis on the GPU*, arXiv:2604.07644.

The purpose of this source is to provide JAX software patterns for SQP, RTI, warm starts, JIT compilation, merit-function line search, and GPU QP solves. Only the generic optimization ideas are required here.

ACI. Gibbs and Candès, *Adaptive Conformal Inference Under Distribution Shift*, NeurIPS 2021, arXiv:2106.00170.

DtACI. Gibbs and Candès, *Conformal Inference for Online Prediction with Arbitrary Distribution Shifts*, JMLR 2024. The authors' implementation is the GitHub repository `isgibbs/DtACI`.

Conformal PID. Angelopoulos, Candès, and Tibshirani, *Conformal PID Control for Time Series Prediction*, arXiv:2307.16895. The authors' implementation is the GitHub repository `aangelopoulos/conformal-time-series`.

Pyrallis. Use the GitHub repository `eladrich/pyrallis` as the reference for nested dataclass configuration, YAML parsing, serialization, and command-line overrides.

Optional reference QP backend. JAXopt's `BoxOSQP` and `OSQP` may be used as a correctness reference or alternative pure-JAX QP implementation. The production solver should nevertheless expose its own stable backend interface so that the QP solver may later be replaced without changing the outer algorithm.

A.14 Definition of done

A first implementation is complete when a single YAML file can select

one of three dependence estimators
+
one of four trajectory signals
+
one outer controller
+
one utility-weight update law
+
full SQP or one-step RTI

without changing experiment code, while preserving the exact resolved-label accounting identity and passing the no-lookahead, telescoping, feasibility, and JIT-equivalence tests above.